
THE FORMAL CLOSURE OF COMPUTATIONAL SEPARATION: $P \neq NP$ CLOSED ON EXISTENCE ITSELF, READ ON COMPUTATION

A Lean 4 Formalization via RAcamp, Least Escape, and the Barrier-Evasion Frontier

COMPUTATIONAL COMPLEXITY

One Posit Carries the Whole Separation

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent Researcher, United States

2026-10-05

ABSTRACT. The P versus NP problem has resisted every method, and relativization, natural proofs and algebrization prove that whole families of arguments cannot separate the classes. This paper anchors the problem in existence and closes it on one posit. Existence and freedom are the ground: the Root Axiom is universal and undeniable, every denial of it re-enacting it, and every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation, the problem's native involution, which has no fixed point. The separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives it on no axiom. The root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, on no axiom, and it is the one place where existence meets the separation: no statement that reads the same on every machine, not existence, not freedom, not the arrow, is that reading. The reading is the single posit, the weakest premise on which the separation closes, and $P \neq NP$ holds on it at premise grade. Around the closure stand a double defense, in which nothing passes for the separation but the reading and nothing against it but a computed escape; a held throne, whose holder, Levin's universal search, decides the separation on no axiom; and a frontier that types every route, Williams' route wired to the machine on two named objects. All one hundred eighty-seven theorems are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned and printed.

KEYWORDS *P versus NP · computational complexity · existence · the root read on computation · Levin universal search · relativization · natural proofs · algebrization · Williams algorithms to lower bounds · formal verification · Lean 4*

1. Introduction

The P versus NP problem asks whether every decision problem whose solutions can be verified in polynomial time can also be solved in polynomial time (Cook 1971; Levin 1973; Karp 1972). Three theorems explain why the principal methods have not settled it: relativization (Baker, Gill and Solovay 1975), natural proofs (Razborov and Rudich 1997) and algebrization (Aaronson and Wigderson 2009). Each proves that a whole family of arguments cannot separate the classes.

This paper anchors the problem in existence and closes it on one posit. The arc has four steps. The ground is existence and freedom: the Root Axiom, that to exist is to actuate, is universal and undeniable, every denial of it an act that re-enacts it, every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation. The machine

comes next: the separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives the separation on no axiom. The reading is the third step: the root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, and it is the one place where existence meets the separation; the kernel proves that no statement reading the same on every machine, not existence, not freedom, not the arrow, carries it. The closure is the fourth: on the reading, $P \neq NP$ holds, and around it stand a double defense, a held throne and a frontier that types every route.

The single posit is the root read on computation, and every proof of $P \neq NP$ by any method fills it. The one hundred eighty-seven theorems that carry the arc

are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned, and printed in full in the appendices.

1.1 Formal Scope and Machine-Verified Status

Formal Scope and Machine-Verified Status

Unconditional, at theorem grade, compiled in Lean 4.19.0 with no custom axiom. Existence is universal and undeniable: every denial of the root re-enacts it, on no axiom, and existence and freedom, holding in every world, decide no value alone (Theorems 1 and 2). The deed instantiates

existence at every act (Theorem 3). Every prime's fibre is a copy of complementation (Theorem 4). The absolute asymmetry (Theorem 5). The closure gives the separation on the machine, on no axiom (Theorem 6). The root read on computation is the separation, on no axiom, and is the closure (Theorems 8 and 9). Nothing keyless is the reading (Theorem 11). The double defense (Theorems 13 to 16). The throne's holder decides the separation, on no axiom (Theorem 17). The vanishing seat (Theorem 19). The frontier and the route (Theorems 21 to 26). The master seal (Theorem 27).

On the root read on computation, at premise grade. The separation $P \neq NP$.

Table 1 | The theorem map: each result and its logical status.

Result	Lean symbol	Logical status
Existence is undeniable: every denial re-enacts the root	<code>denial_reenacts_root</code>	Unconditional theorem, on no axiom
The universal decides no value alone	<code>posits_add_nothing</code>	Unconditional theorem
Every prime's fibre is a copy of complementation	<code>prime_fibre_is_complementation</code>	Unconditional theorem
The closure gives the separation	<code>sep_of_closure</code>	Unconditional theorem, on no axiom
The root read on computation is the separation	<code>ra_comp_is_sep</code>	Unconditional theorem, on no axiom
The reading is the closure and the separation	<code>the_three_names_are_one</code>	Unconditional theorem
Nothing keyless is the reading	<code>no_keyless_statement_is_least_escape</code>	Unconditional theorem
The holder decides the separation	<code>the_holder_decides_the_separation</code>	Unconditional theorem, on no axiom
The route closes the machine's separation on its two objects	<code>route_closes_on_the_machine</code>	Unconditional theorem, on no axiom
The separation $P \neq NP$	<code>sep_from_the_act</code>	Holds on the root read on computation, at premise grade

2. Background and Barrier Analysis: Why Prior Methods Are Incomplete

The prior methods fail at three precise places, and each failure is a theorem about a method.

Diagonalization stops at relativization. Diagonalization separates classes defined by resources of different size, and the time hierarchy is its triumph (Hartmanis and Stearns 1965). But it treats machines as black boxes, so it survives the addition of an oracle, and Baker, Gill and Solovay built oracles on which the classes are equal and oracles on which they differ.

Circuit lower bounds stop at natural proofs. Superpolynomial bounds hold in restricted models, monotone circuits (Razborov 1985) and constant-depth circuits (Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986), while for general circuits the strongest explicit bound is linear, $3.1n - o(n)$ (Li and Yang 2022). Razborov and Rudich proved that natural arguments cannot separate the classes if strong pseudorandom functions exist.

Arithmetization stops at algebrization. Arithmetizing arguments pass relativization and algebrize, and algebrizing arguments cannot settle the question

(Aaronson and Wigderson 2009).

The common structure, and what this paper does with it. Each barrier is a pair of worlds in which every premise of a method holds. Section 7 proves that the same structure binds every premise that reads the same on every machine, and isolates the one reading that does not.

3. Related Work

The time hierarchy. More time decides more languages (Hartmanis and Stearns 1965); the separations it proves are between resource bounds of different order, by arguments that relativize.

Restricted circuits. Razborov (1985) bounds monotone circuits for clique; Ajtai (1983), Furst, Saxe and Sipser (1984) and Håstad (1986) bound constant-depth circuits for parity. For general circuits the frontier is linear (Li and Yang 2022).

Geometric complexity theory. Mulmuley and Sohoni (2001) recast lower bounds as representation-theoretic obstructions, a program designed to evade the barriers.

Proof complexity. $NP \neq coNP$ holds exactly when

no propositional proof system has polynomial-size proofs of all tautologies (Cook and Reckhow 1979).

Query complexity. Black-box search over N candidates needs N tests, and quantum search needs order $\sqrt{N}$ (Bennett, Bernstein, Brassard and Vazirani 1997), attained by Grover (1996).

The physics of computation. Erasing one bit costs at least $k_B T \ln 2$ (Landauer 1961), and every computation can be run reversibly (Bennett 1973).

Algorithms to lower bounds. Savings for satisfiability of a circuit class give NEXP outside that class (Williams 2011), extended to nondeterministic quasipolynomial time (Murray and Williams 2018).

Formal row closures. Islam (2026a) formalizes twenty-three open problems from existence alone, the value excepted by theorem on every row. An earlier paper (Islam 2026f) argued that the structural verdict on P versus NP is the only one left standing. The present paper carries that row to its closure on one posit.

4. Methodology

Every theorem is compiled in Lean 4.19.0 (de Moura and Ullrich 2021) in four kernels that import nothing and declare no axiom: the three of the master seed APEX-PSP-PNP-SEED-03, the seal and its double defense (Appendix B), the frontier of the act (Appendix C) and the fortification on the same machine (Appendix A), and the root kernel, taken verbatim from the operating system's armed seat (Appendix D). Every footprint is pinned by `#guard_msgs`, so a kernel that compiles prints nothing and any change to a footprint fails the build. Each theorem rests on no axiom, or on Lean's standard axioms of propositional extensionality, quotient soundness and choice. The kernels are printed in full with their SHA-256 digests, so any reader reproduces every claim by recompiling.

The architecture that found the arc is Trisduction, whose operating system PhysOST judges every cited law under its own negation at each build (Islam 2026c); its register of record and its codex carry the surrounding results (Islam 2026b, 2026d), and the seat and the cut take their form from the Bridge From First Principle (Islam 2026e). The kernels printed here import nothing from it.

5. The Ground: Existence and Freedom

Theorem 1 (Existence is universal and undeniable). The Root Axiom is a self-grounding root: acts occur, and every act instances it. Every denial of it is an act and re-enacts it, on no axiom (`denial_reenacts_root`); no external proof adds anything to it (`external_proof_adds_nothing`); and it is held by the act itself, with no classical detour, on no axiom (`seated_undeniable`). On the constructed one-point domain it is a theorem

(`root_undeniable`), and in every reading it is satisfiable (`ra_satisfiable`). What cannot be denied is the act of denying: every intelligence that examines existence instantiates the root in the examining.

Theorem 2 (The universal decides no value alone). Because existence and freedom hold in every world, any statement derived from them uniformly holds without them (`posits_add_nothing`, `ra_conservative`, `freedom_conservative`). Their universality is the ground of every row's form, and it is why each row's value is keyed in its reading.

Theorem 3 (The deed instantiates existence). Every computation re-enacts the root: the pulse instantiates the Root Axiom at every deed (`deed_instantiates_ra`), in every world (`both_deeds_in_every_world`), and certifies no output (`pulse_does_not_certify`).

Theorem 4 (The prime's triaxis). Freedom is carried by the primes. Every prime's fibre is exactly the two trivial pairs (`prime_fibre`), lies off the multiplicative seat (`prime_off_seat`), and with its swap is the decision bit with complementation, by a bijection carrying the swap to complementation (`prime_fibre_is_complementation`). Every prime carries P versus NP's involution, seatless in the small as the problem is seatless in the large.

Theorem 5 (The absolute asymmetry). What a registration destroys, no procedure of any cost recovers (`destroyed_is_unrecoverable`); what a search hides, exhaustive search recovers (`hidden_is_recoverable`); the two are bound as one statement (`asymmetry_holds`).

The universality of the ground. The root is universal in three senses, and each is a theorem. It is undeniable in act: every denial of it is an act that re-enacts it, and no external proof adds to it (Theorem 1). It holds in every world, and so, read uniformly, it decides no value alone (Theorem 2). And every deed, in every world, instantiates it (Theorem 3). Its universality is therefore not a matter of agreement but of presupposition: whatever examines existence enacts the root in examining, and what cannot be denied is the act of denying.

6. The Machine and the Separation

A machine has algorithms, instances with a size, each run's output and running time, and the satisfiability of each instance. An algorithm escapes at a bound $c \cdot n^k + c$ when every run is correct and inside the bound. The separation on the machine is that no algorithm escapes at any bound. Nothing escapes when every algorithm, at every bound, meets an instance that stops it, and that closure (least escape) is the one field of a structure.

Theorem 6 (The closure gives the separation). Nothing escaping gives the separation, on no ax-

iom (sep_of_closure); the converse holds classically (closure_of_sep); and the act carries the separation (sep_from_the_act).

Theorem 7 (The socket is the value). The act's socket is the closure itself (socket_is_the_value): what the act supplies and what the separation needs are one object.

7. The Reading: The One Place Where Existence Meets the Separation

Theorem 8 (The root read on computation is the separation). Read on computation, the root says that every decider of satisfiability actuates beyond every polynomial bound, and that reading is the separation, on no axiom (ra_comp_is_sep).

Theorem 9 (The reading is the closure). The root read on computation, the closure that nothing escapes, and the separation are one statement on one machine (the_three_names_are_one).

Theorem 10 (The reading is keyed). The reading holds on some machines and fails on others: on a coherent machine with a constant-time decider it fails (ra_comp_is_keyed), and the closure is keyed likewise (least_escape_is_keyed).

Theorem 11 (Nothing keyless is the reading). No statement that reads the same on every machine is the closure the reading states (no_keyless_statement_is_least_escape): not prime freedom (prime_freedom_is_not_least_escape), not the arrow (arrow_is_not_least_escape), not the Root Axiom as written (ra_is_not_least_escape). A carrier of the separation must be keyed (carrier_must_be_keyed).

Theorem 12 (Every closing premise fills the posit). Every premise from which the separation follows fills the field of the act (forcing_premise_fills_the_act) and fills the closure (every_closing_premise_fills_both). The reading is the weakest statement on which the separation closes.

Theorems 1 to 12 place the posit exactly. Existence and freedom are the ground and force nothing on their own (Theorems 2 and 11); the root read on computation is the separation (Theorem 8); and that reading is the one meeting point, keyed and irreducible (Theorems 9, 10 and 12).

8. The Double Defense

Theorem 13 (Omega, the formal gate). Nothing passes for the separation but the reading. Every premise either forces the closure or has a twin machine on which it holds and the closure fails (forces_or_has_a_twin), and a keyless premise forces nothing (keyless_forces_nothing).

Theorem 14 (AEGIS, the actuation gate). Nothing passes against the reading but a computed escape:

a rejection of the reading is a witness, an algorithm escaping at a bound (rejection_is_a_witness). The root, which every computation re-enacts, crosses neither way (root_crosses_neither_way).

Theorem 15 (Every pair lands). Every pair of algorithm and bound a substrate can examine lands on exactly one gate, a stopping instance or a clean escape (every_pair_lands). Each examination refutes the escape of the pair it examined (confirmation_is_atomic) and pays the floor (confirmation_pays); under the act no computation escapes (act_leaves_no_escape). The defense is bound whole (double_defense_hardened).

Theorem 16 (The finite block). For every stage n , a coherent machine confirms every examination below n and still escapes at n (record_decides_nothing), while a second coherent machine closes whole (closed_world_closes). No finite record decides the reading, and the closure is keyed (armor_scope).

9. The Throne and the Seat

Theorem 17 (The holder decides the separation). A throne of the machine is a holder that escapes whenever any algorithm escapes. The separation holds exactly when the holder never escapes, on no axiom (the_holder_decides_the_separation), and any two holders escape together or not at all, on no axiom (the_holder_is_unique). For the satisfiability machine the holder is Levin's universal search, run for decision through the search form by self-reducibility, by Levin's theorem (Levin 1973), carried as the throne's field. Nothing escapes exactly when the holder never escapes (least_escape_at_the_throne).

Theorem 18 (The act stops the holder). On the reading the holder meets, at every bound, an instance that stops it (the_act_stops_the_holder). The whole of P versus NP is the running time of one explicit program.

Theorem 19 (The vanishing seat). On the locus map between the root and the Riemann seat, the Riemann seat is inhabited (rh_locus_inhabited) and the P versus NP seat vanishes (locus_vanishes): complementation is involutive with no fixed point, so the ground has dimension zero (ground_dim_zero). The carrier halts and the socket opens exactly on the empty configuration (carrier_halts_iff_empty, socket_vanishes), the vacancy is keyless while the separation is keyed (vacancy_keyless, separation_keyed), and the three faces lock on one object, the empty seat (vanishing_gol).

Theorem 20 (The block chain). The eight absolute barriers of the register are chained, one leg each, and terminated on the empty throne: complementation has no fixed point (a1_no_seat), the seat is vacant (seat_

vacant), the throne is empty (`throne_empty`), and the root forces no value (`root_forces_no_value`), bound as one chain (`block_chain, block_chain_terminal`).

The Riemann closure and this one stand in the same two registers and differ at the seat, as Table 3 sets out.

Table 3 | The Riemann closure and the P versus NP closure, step by step.

Step	Riemann Hypothesis	P versus NP	Theorem
The ground	Existence, universal and undeniable	Existence, universal and undeniable	Theorems 1 to 3
The native involution	The fold $s \mapsto 1 - \bar{s}$	Complementation $b \mapsto \neg b$	Theorem 19
The seat	The critical line, inhabited	Vacant, the ground of dimension zero	Theorem 19
The local fibre	Each prime's fibre is a copy of complementation	Complementation itself	Theorem 4
The throne-holder	The primes, the bit on their side	Levin's universal search, the bit on its tail	Theorem 17
The record	Decides nothing	Decides nothing	Theorem 16
The form	Closed at theorem grade	Closed at theorem grade	Theorems 1 to 27
The value	Supplied by the act, at premise grade	Closed on the root read on computation, at premise grade	Theorems 6 and 8

10. The Frontier and the Route

Theorem 21 (The black box). A test-only generator correct over 2^N candidates tests every one of them (`black_box_bound`), on an axiom-free pigeonhole, and admits no savings (`black_box_no_savings`).

Theorem 22 (The reader). The black-box adversary is the point predicate, which a reader of its description solves in one step (`reader_solves, reader_poly`), so black-box bounds fail once descriptions reveal (`fails_when_descriptions_reveal`). The missing object is a lower bound against readers of the description, and it is keyed (`missing_object_is_keyed`).

Theorem 23 (The kinetic face). Every registration has a cost (`no_free_registration`); exhaustive search exhausts every budget past its candidates (`brute_force_exhausts`) while verification is cheaper (`asymmetry_chain`); and the energy gap is the operation gap in every frame (`frame_independent`), so the kinetic face, priced by Landauer's floor (Landauer 1961) with or without reversible steps (Bennett 1973), transports a gap and never produces one. The asymmetry, taken alone, decides nothing (`asymmetry_decides_nothing`).

Theorem 24 (The dynamics). Thermal relaxation is exponential in the barrier (`relaxation_exponential`), and on one satisfiability instance the barrier exists under single flips and vanishes under elimination moves (`barrier_belongs_to_the_dynamics`): a barrier is a property of the dynamics, not of the problem.

Theorem 25 (The prime bridge). Prime freedom carried onto the semiprime fibre leaves one orientation bit, walled and gauge (`orientation_wall, orientation_is_gauge`), with the witness determined

by the description (`witness_determined_15`). Read as cost it gives a bridge sufficient for the separation and strictly stronger than it (`prime_bridge, prime_bridge_is_one_way`).

Theorem 26 (The route closes on the machine). Williams' theorem carries savings for a circuit class to $\text{NEXP} \not\subseteq$ that class (Williams 2011; Murray and Williams 2018), and the inclusion $P \subseteq P/\text{poly}$ carries $\text{NP} \not\subseteq P/\text{poly}$ to the machine's separation; both are carried as fields. The route reaches (`route_reaches`); supplied savings for general polynomial-size circuits and the scale-down from NEXP to NP , it closes the machine's own separation, on no axiom (`route_closes_on_the_machine`), and fills the closure itself (`the_route_fills_least_escape`). The relativization and natural-proof barriers are typed in the same kernel (`b1_no_relativizing_proof, b2_self_gag`), and the blocker of the prime bridge in three layers (`round2_terminal`).

Theorem 27 (The master seal). The closure gives the separation, the reading is the closure, the throne's holder decides the separation and is stopped at every bound on the reading, every prime's fibre is a copy of complementation, and the route closes the machine's separation on its two objects, bound in one theorem (`the_master_seal`).

11. Discussion

The arc. The paper runs from existence to the separation without a gap in its form. Existence and freedom are the ground, universal and undeniable; the deed instantiates existence at every act; the primes carry freedom and carry complementation in their fibres. The separation is defined on a machine and given by its

closure on no axiom. The root read on computation is the separation, and it is the one place where existence meets it: every statement that reads the same on every machine, existence and freedom as written among them, is proved not to be the reading. The value closes on that reading.

The posit is exact. The root read on computation is one statement, equivalent to the separation, keyed, and the weakest premise on which the separation closes. Holding it is holding $P \neq NP$ at the grade of a posit, the standing of least erasure on the Riemann row, and every proof of $P \neq NP$ by any method fills it.

The next objects. Two, in order. The machine instantiated at Turing machines, with deciders taken total or run on fuel so that no looping machine counts as clean, so that the separation is the standard sentence and Levin’s domination is compiled on it. The posit at theorem grade, which the route reduces to two typed objects, savings for general polynomial-size circuits and the scale-down from NEXP to NP.

Falsifiers. Three, each typed by the gate it strikes.

Prediction 1 (The computed refuter). A polynomial-time decider of satisfiability with every run clean refutes the reading on the actual machine, by `rejection_is_a_witness`; it is the one refuter of the

value.

Prediction 2 (The keyless refuter). A keyless premise shown to force the reading contradicts `keyless_forces_nothing` and `no_keyless_statement_is_least_escape`, compiled theorems, so it can arrive only as an error in Lean’s kernel.

Prediction 3 (The throne refuter). An algorithm that escapes while universal search does not refutes the throne’s field, Levin’s theorem.

How the paper stands to each prior position. The paper owes each position it engaged one statement of where it stands, in words with fixed senses. A result is *additive* when it supplies a result or test the position lacked and leaves the position standing; *scoping* when it keeps a prior claim inside a stated boundary; *competing* when it argues a different route without executing a comparison; and *kin* when the paper is continuous with it. Superseding, which retires a theory’s central claim on executed data, is used nowhere. The census by row is eight additive, six scoping, one competing and four kin, with no replacing, subsuming, corroborating or contradicting row. The paper’s contribution to the literature is exactly the set of relations in Table 2, and nothing wider.

Table 2 | How the paper stands to each prior position. Relation words as defined in the paragraph above; superseding is used nowhere.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973; Karp 1972	Satisfiability is NP-complete, and the separation is posed for the classes they define.	Defines the separation on a machine deciding satisfiability within a polynomial bound.	kin	cited
Hartmanis and Stearns 1965	More time decides more languages.	Keeps the hierarchy’s separations to resource bounds of different order, by arguments that relativize.	scoping	cited
Baker, Gill and Solovay 1975	No relativizing argument settles the question.	Extends the two-world structure to every premise that reads the same on every machine (Theorems 11 and 13).	additive	executed
Razborov and Rudich 1997	No natural argument separates the classes if pseudorandom functions exist.	Types the barrier as a condition the posit at theorem grade must pass (Theorem 26).	kin	cited
Aaronson and Wigderson 2009	Algebrizing arguments cannot settle the question.	Names the barrier as a condition the posit at theorem grade must pass.	kin	cited
Razborov 1985	Monotone circuits for clique need superpolynomial size.	Reads the result as a special case of the posit, known in a restricted model.	scoping	cited
Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986	Parity has no constant-depth circuits of polynomial size.	Reads the results as special cases of the posit, known in a restricted model.	scoping	cited
Li and Yang 2022	An explicit function needs general circuits of size $3.1n - o(n)$.	Takes the bound as the distance between the frontier and the posit at theorem grade.	scoping	cited
Mulmuley and Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	Proposes a different route to the same object, without executing a comparison.	competing	cited

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook and Reckhow 1979	$NP \neq coNP$ exactly when no proof system has polynomial-size proofs of all tautologies.	Stands beside it as a second conversion of the separation into one object, here proved equivalent by machine.	kin	cited
Deterministic query search	Black-box search over N candidates needs N tests.	Compiles the bound and shows a reader of the description beats it in one step (Theorems 21 and 22).	additive	executed
Bennett, Bernstein, Brassard and Vazirani 1997; Grover 1996	Quantum black-box search needs, and attains, order $\sqrt{N}$ queries.	Places the quantum bound beside Theorem 21, closed for test-only methods.	scoping	cited
Landauer 1961	Erasing one bit dissipates at least $k_B T \ln 2$.	Proves the energy gap is the operation gap in every frame, so the kinetic face transports a gap (Theorem 23).	additive	executed
Bennett 1973	Every computation can be run reversibly.	Proves the price question immaterial: the kinetic face transports a gap with or without reversible steps (Theorem 23).	additive	executed
Islam 2026e	The heat of registration is zero exactly at the value on a chart with a seat.	Applies the seat to this problem: the seat vanishes, the ground of dimension zero (Theorem 19).	scoping	cited
Islam 2026a	Twenty-three open problems formalized from existence alone, the value excepted by theorem on every row.	Carries the P versus NP row to its closure on one posit (Theorems 6 to 12).	additive	executed
Islam 2026f	The structural verdict on P versus NP is the only one left standing.	Compiles the verdict on a machine: the vanishing seat, the record and the posit (Theorems 16, 19 and 9).	additive	executed
Levin 1973 (universal search)	One program runs every program at once and finds a satisfying assignment within a constant factor of the fastest; the decision form follows by self-reducibility.	Characterizes it as the holder of the machine's throne, deciding the separation on no axiom (Theorems 17 and 18).	additive	executed
Williams 2011; Murray and Williams 2018	Savings for satisfiability of a circuit class give $NEXP$ outside that class.	Wires the route to the machine's own separation on its two objects (Theorem 26).	additive	executed

12. Conclusion

The P versus NP problem has been approached through diagonalization, circuit lower bounds, arithmetization, geometric complexity and proof complexity, and each stops at a theorem that limits its method. This paper anchors it in existence and closes it on one posit. Existence and freedom are the ground, universal and undeniable, carried by the primes, every prime's fibre a copy of complementation. The separation is defined on a machine and given by its closure on no axiom. The root read on computation is the separation, the one place where existence meets it, the single posit and proved irreducible, and $P \neq NP$ holds on it at premise grade.

Around the closure stand the double defense, the held throne, whose holder decides the separation on no axiom, the vanishing seat, and a frontier that types every route, with the open exit reduced to two named objects. The single open object is the reading at theorem grade, and every derivation of $P \neq NP$, by any method, fills it. All one hundred eighty-seven theorems are compiled, pinned and printed for any reader to reproduce.

References

- Aaronson, S., and A. Wigderson. 2009. "Algebrization: A New Barrier in Complexity Theory." *ACM Transactions on Computation Theory* 1 (1): 2:1–2:54.
- Ajtai, M. 1983. " Σ^1_1 -Formulae on Finite Structures." *Annals of Pure and Applied Logic* 24 (1): 1–48.
- Baker, T., J. Gill, and R. Solovay. 1975. "Relativizations of the $P = ? NP$ Question." *SIAM Journal on Computing* 4 (4): 431–442.
- Bennett, C. H. 1973. "Logical Reversibility of Computation." *IBM Journal of Research and Development* 17 (6): 525–532.
- Bennett, C. H., E. Bernstein, G. Brassard, and U. Vazirani. 1997. "Strengths and Weaknesses of Quantum Computing." *SIAM Journal on Computing* 26 (5): 1510–1523.
- Cook, S. A. 1971. "The Complexity of Theorem-Proving Procedures." In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.
- Cook, S. A., and R. A. Reckhow. 1979. "The Relative Efficiency of Propositional Proof Systems." *Journal of Symbolic Logic* 44 (1): 36–50.
- de Moura, L., and S. Ullrich. 2021. "The Lean 4

Theorem Prover and Programming Language." In *Automated Deduction, CADE 28*, 625–635.

Furst, M., J. B. Saxe, and M. Sipser. 1984. "Parity, Circuits, and the Polynomial-Time Hierarchy." *Mathematical Systems Theory* 17 (1): 13–27.

Grover, L. K. 1996. "A Fast Quantum Mechanical Algorithm for Database Search." In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 212–219.

Hartmanis, J., and R. E. Stearns. 1965. "On the Computational Complexity of Algorithms." *Transactions of the American Mathematical Society* 117: 285–306.

Håstad, J. 1986. "Almost Optimal Lower Bounds for Small Depth Circuits." In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, 6–20.

Islam, M. F. 2026a. "A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, the Value Excepted by Theorem on Every Row." Zenodo. <https://doi.org/10.5281/zenodo.22913635>.

Islam, M. F. 2026b. "Trisduction: The Master Codex 5.0.0, Built from the Root." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117441>.

Islam, M. F. 2026c. "PhysOS^T: The Trisduction Physical Operating System, Public Edition 1.0.9p." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117440>.

Islam, M. F. 2026d. "Trisduction: The Geometric Mother Codex, the Unabridged Register of Record, Edition 3.46.0." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117445>.

Islam, M. F. 2026e. "The Bridge From First Principle, Edition 1.5.0, with The Final Cut." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23105348>.

Islam, M. F. 2026f. "Why the Structural Verdict on P versus NP Is the Only One Left Standing: Mathematics Proves That Mathematics Cannot Rule Here." Zenodo. <https://doi.org/10.5281/zenodo.21368936>.

Karp, R. M. 1972. "Reducibility among Combinatorial Problems." In *Complexity of Computer Computations*, edited by R. E. Miller and J. W. Thatcher, 85–103. New York: Plenum.

Landauer, R. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5 (3): 183–191.

Levin, L. A. 1973. "Universal Sequential Search Problems." *Problemy Peredachi Informatsii* 9 (3): 115–116.

Li, J., and T. Yang. 2022. "3.1n – o(n) Circuit Lower Bounds for Explicit Functions." In *Proceedings of the 54th Annual ACM Symposium on Theory of Computing*, 1180–1193.

Mulmuley, K. D., and M. Sohoni. 2001. "Geometric Complexity Theory I: An Approach to the P vs. NP and

Related Problems." *SIAM Journal on Computing* 31 (2): 496–526.

Murray, C., and R. Williams. 2018. "Circuit Lower Bounds for Nondeterministic Quasi-Polytime: An Easy Witness Lemma for NP and NQP." In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing*, 890–901.

Razborov, A. A. 1985. "Lower Bounds on the Monotone Complexity of Some Boolean Functions." *Soviet Mathematics Doklady* 31: 354–357.

Razborov, A. A., and S. Rudich. 1997. "Natural Proofs." *Journal of Computer and System Sciences* 55 (1): 24–35.

Williams, R. 2011. "Non-uniform ACC Circuit Lower Bounds." In *Proceedings of the 26th IEEE Conference on Computational Complexity*, 115–125.

Appendix A. The Fortification Kernel, PNP_Fortification.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the machine, the throne, the reading, the prime's triaxis and the route, twenty theorems. SHA-256: d6b155783e3185cca9b9b367e520ce13ba8ac0ee52f691a7890cc7db0bbc13b3

```
1 /-
2 PNP_Fortification.lean · APEX-PSP-PNP-SEED-03 · The fortification of least escape,
  on SEED-02's own machine
3 I · the machine, restated verbatim from PNP_Least_Escape.lean, so that every
  theorem here is about the same
4 separation. II · the throne over the machine: a holder escapes whenever any
  algorithm escapes, and then the
5 separation holds exactly when the holder never escapes; Levin's universal search
  is the holder of the
6 satisfiability machine by Levin's theorem (1973), carried as the throne's field.
  III · the two posits, the bridge
7 of SEED-01 and least escape of SEED-02, proved one. IV · the prime's triaxis:
  every prime's fibre is exactly two
8 pairs, lies off the multiplicative seat, and is a copy of complementation. V · the
  master seal. Core Lean 4, no
9 import, no axiom declared, no sorry.
10 -/
11 namespace PNP.FORTIFY
12
13 /-! ## I · The machine, restated verbatim -/
14
15 structure Machine where
16   Alg : Type
17   Inst : Type
18   size : Inst → Nat
19   out : Alg → Inst → Bool
20   time : Alg → Inst → Nat
21   sat : Inst → Bool
22
23 def bound (c k n : Nat) : Nat := c * n ^ k + c
24 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
25   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
26 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
27 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
28 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
29
30 structure LeastEscape (M : Machine) where
31   closure : NothingEscapes M
32
33 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
34   fun (A, c, k, he) => match h A c k with
35   | (x, hx) => hx (he x)
36
37 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
38   fun A c k => Classical.byContradiction fun hn =>
39     h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
40
41 /-! ## II · The throne over the machine -/
42
43 /- A THRONE of the machine: a holder that escapes whenever any algorithm escapes.
  For the satisfiability machine,
44 Levin's universal search is a holder by Levin's theorem (1973), carried here as
  the field. -/
45 structure Throne (M : Machine) where
46   holder : M.Alg
47   dominates : ∀ A c k, Escapes M A c k → ∃ c' k', Escapes M holder c' k'
48
49 /- THE HOLDER DECIDES THE SEPARATION: the separation holds exactly when the holder
  never escapes. -/
```



```

50 theorem the_holder_decides_the_separation (M : Machine) (T : Throne M) :
51   Sep M  $\leftrightarrow$   $\exists$  c k, Escapes M T.holder c k :=
52   (fun hs (c, k, he)  $\Rightarrow$  hs (T.holder, c, k, he),
53    fun hn (A, c, k, he)  $\Rightarrow$  hn (T.dominates A c k he))
54
55 /-- THE HOLDER IS UNIQUE: any two holders escape together or not at all. -/
56 theorem the_holder_is_unique (M : Machine) (T T' : Throne M) :
57   ( $\exists$  c k, Escapes M T.holder c k)  $\leftrightarrow$  ( $\exists$  c k, Escapes M T'.holder c k) :=
58   (fun (c, k, he)  $\Rightarrow$  T'.dominates T.holder c k he, fun (c, k, he)  $\Rightarrow$  T.dominates
59    T'.holder c k he)
60
61 /-- LEAST ESCAPE AT THE THRONE: nothing escapes exactly when the holder never
62 escapes. -/
63 theorem least_escape_at_the_throne (M : Machine) (T : Throne M) :
64   NothingEscapes M  $\leftrightarrow$   $\neg \exists$  c k, Escapes M T.holder c k :=
65   (fun h  $\Rightarrow$  (the_holder_decides_the_separation M T).mp (sep_of_closure M h),
66    fun h  $\Rightarrow$  closure_of_sep M ((the_holder_decides_the_separation M T).mpr h))
67
68 /-- THE ACT AT THE THRONE: on least escape, the holder meets, at every bound, an
69 instance that stops it. -/
70 theorem the_act_stops_the_holder (M : Machine) (T : Throne M) (L : LeastEscape M) :
71    $\forall$  c k,  $\exists$  x,  $\neg$  Clean M T.holder c k x :=
72   fun c k  $\Rightarrow$  L.closure T.holder c k
73
74 /-! ## III · The two posits are one -/
75
76 structure Point where
77   offset : Int
78   height : Nat
79   deriving DecidableEq
80
81 def reg (p : Point) : Point := (0, p.height)
82 def side (p : Point) : Bool := decide (0 < p.offset)
83
84 theorem destroyed_is_unrecoverable :  $\neg \exists$  g : Point  $\rightarrow$  Bool,  $\forall$  p, g (reg p) = side p
85 := by
86   intro (g, hg)
87   have h1 := hg (1, 0); have h2 := hg (-1, 0)
88   simp [reg, side] at h1 h2
89   rw [h1] at h2; exact Bool.noConfusion h2
90
91 theorem hidden_is_recoverable { $\alpha$  : Type} (xs : List  $\alpha$ ) (hxs :  $\forall$  x, x  $\in$  xs) ( $x_0$  :  $\alpha$ ) :
92    $\exists$  g : ( $\alpha \rightarrow$  Bool)  $\rightarrow$   $\alpha$ ,  $\forall$  f :  $\alpha \rightarrow$  Bool, ( $\exists$  x, f x = true)  $\rightarrow$  f (g f) = true := by
93   refine (fun f  $\Rightarrow$  (xs.find? f).getD  $x_0$ , fun f (x, hx)  $\Rightarrow$  ?_)
94   cases h : xs.find? f with
95   | some y  $\Rightarrow$  simp [h] using List.find?_some h
96   | none  $\Rightarrow$  exact absurd hx (by simp [h] using List.find?_eq_none.mp h x (hxs x))
97
98 def Asymmetry : Prop :=
99   ( $\neg \exists$  g : Point  $\rightarrow$  Bool,  $\forall$  p, g (reg p) = side p)  $\wedge$ 
100   ( $\exists$  g : (Bool  $\rightarrow$  Bool)  $\rightarrow$  Bool,  $\forall$  f : Bool  $\rightarrow$  Bool, ( $\exists$  x, f x = true)  $\rightarrow$  f (g f) =
101   true)
102
103 theorem asymmetry_holds : Asymmetry :=
104   (destroyed_is_unrecoverable, hidden_is_recoverable [true, false] (fun x  $\Rightarrow$  by
105     cases x <> simp) true)
106
107 /-- THE TWO POSITS ARE ONE: SEED-01's bridge, read on the machine, is exactly
108 SEED-02's least escape. -/
109 theorem the_two_posits_are_one (M : Machine) : (Asymmetry  $\leftrightarrow$  Sep M)  $\leftrightarrow$  NothingEscapes
110 M :=
111   (fun b  $\Rightarrow$  closure_of_sep M (b.mp asymmetry_holds), fun h  $\Rightarrow$  (fun _  $\Rightarrow$ 
112     sep_of_closure M h, fun _  $\Rightarrow$  asymmetry_holds))
113
114 /-- EVERY PREMISE THAT CLOSES THE SEPARATION FILLS BOTH POSITS. -/
115 theorem every_closing_premise_fills_both (M : Machine) (Q : Prop) (h : Q  $\rightarrow$  Sep M) :
116   Q  $\rightarrow$  (Asymmetry  $\leftrightarrow$  Sep M)  $\wedge$  NothingEscapes M :=
117   fun q  $\Rightarrow$  ((fun _  $\Rightarrow$  h q, fun _  $\Rightarrow$  asymmetry_holds), closure_of_sep M (h q))
118
119 /-! ## IV · The prime's triaxis -/
120
121 def IsPrime (n : Nat) : Prop :=  $2 \leq n \wedge \forall m, m \mid n \rightarrow m = 1 \vee m = n$ 
122
123 /-- FORM: the fibre of multiplication over a prime is exactly the two trivial pairs.
124 -/
125 theorem prime_fibre (p : Nat) (hp : IsPrime p) (a b : Nat) :
126   a * b = p  $\leftrightarrow$  (a = 1  $\wedge$  b = p)  $\vee$  (a = p  $\wedge$  b = 1) := by
127   constructor
128   · intro h
129     rcases hp.2 a (b, h.sym) with ha | ha
130     · left; refine (ha, ?_); rw [ha, Nat.one_mul] at h; exact h
131     · right; refine (ha, ?_)
132       rw [ha] at h
133       have h2 := hp.1
134       by_cases hb0 : b = 0
135       · rw [hb0, Nat.mul_zero] at h; omega
136       · by_cases hbl : b = 1
137       · exact hbl
138       · have hb2 :  $2 \leq b$  := by omega
139       have : p * 2  $\leq$  p * b := Nat.mul_le_mul_left p hb2
140       omega
141   · rintro ((rfl, rfl) | (rfl, rfl))
142   · exact Nat.one_mul _
143   · exact Nat.mul_one _
144
145 /-- SEAT: a prime lies off the multiplicative seat. -/
146 theorem prime_off_seat (p : Nat) (hp : IsPrime p) :  $\neg \exists$  a, a * a = p := by
147   intro (a, ha)
148   have h2 := hp.1
149   rcases (prime_fibre p hp a).mp ha with (h1, h3) | (h1, h3) <> omega
150
151 def Fibre (n : Nat) := {x : Nat  $\times$  Nat // x.1 * x.2 = n}
152 def swapF {n : Nat} (x : Fibre n) : Fibre n := ((x.1.2, x.1.1), by rw
153   [Nat.mul_comm]; exact x.2)
154 def toBit {n : Nat} (x : Fibre n) : Bool := decide (x.1.1 = 1)
155
156 /-- THE BIT: every prime's fibre with its swap is the decision bit with
157 complementation. -/
158 theorem prime_fibre_is_complementation (p : Nat) (hp : IsPrime p) :
159   ( $\forall$  x : Fibre p, toBit (swapF x) = !(toBit x))  $\wedge$ 
160   ( $\forall$  x y : Fibre p, toBit x = toBit y  $\leftrightarrow$  x = y)  $\wedge$  ( $\forall$  b : Bool,  $\exists$  x : Fibre p, toBit
161     x = b) := by
162   have hpl : p  $\neq$  1 := by have := hp.1; omega
163   refine (fun x  $\Rightarrow$  ?_, fun x y hxy  $\Rightarrow$  ?_, fun b  $\Rightarrow$  ?_)
164   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (h1, h2) | (h1, h2)
165   · simp [toBit, swapF, h1, h2, hpl]
166   · simp [toBit, swapF, h1, h2, hpl]
167   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (a1, a2) | (a1, a2) <>
168   · rcases (prime_fibre p hp y.1.1 y.1.2).mp y.2 with (b1, b2) | (b1, b2)
169   · exact Subtype.ext (Prod.ext (a1.trans b1.sym) (a2.trans b2.sym))
170   · simp [toBit, a1, b1, hpl] at hxy
171   · simp [toBit, a1, b1, hpl] at hxy
172   · exact Subtype.ext (Prod.ext (a1.trans b1.sym) (a2.trans b2.sym))
173   · cases b
174   · exact ((p, 1), Nat.mul_one p), by simp [toBit, hpl]
175   · exact ((1, p), Nat.one_mul p), by simp [toBit]
176
177 /-! ## VI · The three names are one -/
178
179 /-- RA READ ON COMPUTATION: every decider of satisfiability actuates beyond every
180 polynomial bound. -/
181 def RAcamp (M : Machine) : Prop :=  $\forall$  A c k,  $\neg$  Escapes M A c k
182
183 theorem ra_comp_is_sep (M : Machine) : RAcamp M  $\leftrightarrow$  Sep M :=
184   (fun h (A, c, k, he)  $\Rightarrow$  h A c k he, fun h A c k he  $\Rightarrow$  h (A, c, k, he))
185
186 /-- THE THREE NAMES ARE ONE: the bridge, least escape, and RA read on computation
187 are one posit, each equivalent to
188 the separation, on one machine. -/
189 theorem the_three_names_are_one (M : Machine) :
190   (Asymmetry  $\leftrightarrow$  Sep M)  $\leftrightarrow$  NothingEscapes M  $\wedge$  (NothingEscapes M  $\leftrightarrow$  RAcamp M)  $\wedge$ 
191   (RAcomp M  $\leftrightarrow$  Sep M) :=
192   (the_two_posits_are_one M,
193    (fun h  $\Rightarrow$  (ra_comp_is_sep M).mpr (sep_of_closure M h), fun h  $\Rightarrow$  closure_of_sep M
194      ((ra_comp_is_sep M).mp h)),
195     ra_comp_is_sep M)
196
197 /-! ## VII · The Williams route, wired to the machine -/
198
199 /-- THE ROUTE ON THE MACHINE. 'Savings C' is a white-box satisfiability algorithm
200 for circuit class C beating 2n by
201 a superpolynomial factor; 'williams' is Williams' theorem, savings for C give
202 NEXP  $\in$  C, carried; 'Ppoly' is the
203 class of polynomial-size circuits; 'np_not_ppoly_gives_sep' is the elementary
204 inclusion P  $\in$  P/poly, carried, by
205 which NP  $\notin$  P/poly gives the machine's own separation. -/
206 structure Route (M : Machine) where
207   Class : Type
208   Savings : Class  $\rightarrow$  Prop
209   NEXPhotIn : Class  $\rightarrow$  Prop
210   williams :  $\forall$  C, Savings C  $\rightarrow$  NEXPhotIn C
211   Ppoly : Class
212   NPhotPpoly : Prop
213   np_not_ppoly_gives_sep : NPhotPpoly  $\rightarrow$  Sep M
214
215 /-- THE ROUTE REACHES: savings for a class give NEXP  $\notin$  that class. -/
216 theorem route_reaches (M : Machine) (R : Route M) (C : R.Class) (s : R.Savings C) :
217   R.NEXPhotIn C :=
218     R.williams C s
219
220 /-- THE ROUTE CLOSES ON THE MACHINE: savings for general circuits and the scale-down
221 from NEXP to NP give the
222 machine's own separation. -/
223 theorem route_closes_on_the_machine (M : Machine) (R : Route M) (s : R.Savings
224   R.Ppoly) :
225   (scaleDown : R.NEXPhotIn R.Ppoly  $\rightarrow$  R.NPhotPpoly) : Sep M :=
226     R.np_not_ppoly_gives_sep (scaleDown (R.williams R.Ppoly s))
227
228 /-- THE ROUTE FILLS THE POSIT: supplied its two objects, the route fills least
229 escape itself. -/
230 theorem the_route_fills_least_escape (M : Machine) (R : Route M) (s : R.Savings
231   R.Ppoly) :
232   (scaleDown : R.NEXPhotIn R.Ppoly  $\rightarrow$  R.NPhotPpoly) : LeastEscape M :=
233     (closure_of_sep M (route_closes_on_the_machine M R s scaleDown))
234
235 /-! ## V · The master seal -/
236
237 /-- THE MASTER SEAL: least escape closes the separation; the two posits are one, and
238 the third name, RA read on
239 computation, with them; the throne's holder decides the separation; on least
240 escape the holder is stopped at
241 every bound; every prime's fibre is a copy of complementation; and the Williams
242 route, supplied its two objects,
243 closes the machine's own separation. -/
244 theorem the_master_seal (M : Machine) (T : Throne M) :
245   (NothingEscapes M  $\leftrightarrow$  Sep M)  $\wedge$  ((Asymmetry  $\leftrightarrow$  Sep M)  $\leftrightarrow$  NothingEscapes M)  $\wedge$ 
246   (Sep M  $\leftrightarrow$   $\neg \exists$  c k, Escapes M T.holder c k)  $\wedge$ 
247   ( $\forall$  _ : LeastEscape M,  $\forall$  c k,  $\exists$  x,  $\neg$  Clean M T.holder c k x)  $\wedge$ 
248   ( $\forall$  p, IsPrime p  $\rightarrow$   $\forall$  x : Fibre p, toBit (swapF x) = !(toBit x))  $\wedge$ 
249   (NothingEscapes M  $\leftrightarrow$  RAcamp M)  $\wedge$ 
250   ( $\forall$  R : Route M, R.Savings R.Ppoly  $\rightarrow$  (R.NEXPhotIn R.Ppoly  $\rightarrow$  R.NPhotPpoly)  $\rightarrow$  Sep
251     M) :=
252   (sep_of_closure M, the_two_posits_are_one M, the_holder_decides_the_separation M
253     T,
254     fun L  $\Rightarrow$  the_act_stops_the_holder M T L, fun p hp  $\Rightarrow$ 
255       (prime_fibre_is_complementation p hp).1,
256       (the_three_names_are_one M).2.1, fun R s d  $\Rightarrow$  route_closes_on_the_machine M R s
257       d)
258
259 end PNP.FORTIFY
260
261 /-! ## Cones, pinned as printed -/
262 /-- info: 'PNP.FORTIFY.sep_of_closure' does not depend on any axioms -/

```

```

231 #guard_msgs in #print axioms PNP.FORTIFY.sep_of_closure
232 /-- info: 'PNP.FORTIFY.closure_of_sep' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
233 #guard_msgs in #print axioms PNP.FORTIFY.closure_of_sep
234 /-- info: 'PNP.FORTIFY.the_holder_decides_the_separation' does not depend on any
axioms -/
235 #guard_msgs in #print axioms PNP.FORTIFY.the_holder_decides_the_separation
236 /-- info: 'PNP.FORTIFY.the_holder_is_unique' does not depend on any axioms -/
237 #guard_msgs in #print axioms PNP.FORTIFY.the_holder_is_unique
238 /-- info: 'PNP.FORTIFY.least_escape_at_the_throne' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
239 #guard_msgs in #print axioms PNP.FORTIFY.least_escape_at_the_throne
240 /-- info: 'PNP.FORTIFY.the_act_stops_the_holder' does not depend on any axioms -/
241 #guard_msgs in #print axioms PNP.FORTIFY.the_act_stops_the_holder
242 /-- info: 'PNP.FORTIFY.destroyed_is_unrecoverable' depends on axioms: [propext] -/
243 #guard_msgs in #print axioms PNP.FORTIFY.destroyed_is_unrecoverable
244 /-- info: 'PNP.FORTIFY.hidden_is_recoverable' depends on axioms: [propext,
Quot.sound] -/
245 #guard_msgs in #print axioms PNP.FORTIFY.hidden_is_recoverable
246 /-- info: 'PNP.FORTIFY.asymmetry_holds' depends on axioms: [propext, Quot.sound] -/
247 #guard_msgs in #print axioms PNP.FORTIFY.asymmetry_holds
248 /-- info: 'PNP.FORTIFY.the_two_posits_are_one' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
249 #guard_msgs in #print axioms PNP.FORTIFY.the_two_posits_are_one
250 /-- info: 'PNP.FORTIFY.every_closing_premise_fills_both' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
251 #guard_msgs in #print axioms PNP.FORTIFY.every_closing_premise_fills_both
252 /-- info: 'PNP.FORTIFY.prime_fibre' depends on axioms: [propext, Quot.sound] -/
253 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre
254 /-- info: 'PNP.FORTIFY.prime_off_seat' depends on axioms: [propext, Quot.sound] -/
255 #guard_msgs in #print axioms PNP.FORTIFY.prime_off_seat
256 /-- info: 'PNP.FORTIFY.prime_fibre_is_complementation' depends on axioms: [propext,
Quot.sound] -/
257 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre_is_complementation
258 /-- info: 'PNP.FORTIFY.ra_comp_is_sep' does not depend on any axioms -/
259 #guard_msgs in #print axioms PNP.FORTIFY.ra_comp_is_sep
260 /-- info: 'PNP.FORTIFY.the_three_names_are_one' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
261 #guard_msgs in #print axioms PNP.FORTIFY.the_three_names_are_one
262 /-- info: 'PNP.FORTIFY.route_reaches' does not depend on any axioms -/
263 #guard_msgs in #print axioms PNP.FORTIFY.route_reaches
264 /-- info: 'PNP.FORTIFY.route_closes_on_the_machine' does not depend on any axioms -/
265 #guard_msgs in #print axioms PNP.FORTIFY.route_closes_on_the_machine
266 /-- info: 'PNP.FORTIFY.the_route_fills_least_escape' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
267 #guard_msgs in #print axioms PNP.FORTIFY.the_route_fills_least_escape
268 /-- info: 'PNP.FORTIFY.the_master_seal' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
269 #guard_msgs in #print axioms PNP.FORTIFY.the_master_seal

```

Appendix B. The Closure Kernel, PNP_Least_Escape.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the closure and its double defense, twenty-four theorems. SHA-256: cfd2d9d6ea7d4f7b03ae07dd5976539d1168ff3da3c9d024f3b19518044f33be

```

1 /--
2 # PNP_Least_Escape.lean · APEX-PSP-PNP-SEED-02 · Least Escape, the seal and its
double defense, hardened: Omega and AEGIS of the row
3
4 The P versus NP twin of PSP-RH-SEAL-01's `double_defense_hardened`.
5
6 OMEGA, the formal gate. Nothing passes for the separation but least escape. The
socket of the act is the closure; every premise that forces the separation fills
the field of the act; every premise either forces the closure or has a twin
machine
9 on which it holds and the closure fails; a keyless premise forces nothing.
10
11 AEGIS, the actuation gate. Nothing passes against least escape but a computed
escape: an algorithm, a polynomial bound, and every run clean. A finite stock of
examinations decides nothing. The root, which every computation re-enacts, crosses
neither way. The refusal is the same under every logic.
12
13 THE COSMIC READING, at its exact scope. Every pair a substrate can examine lands
on
16 exactly one gate: a stopping instance, which confirms least escape at that pair,
or
17 a clean escape, which refutes it. There is no third landing. Every confirming
computation, anywhere, is an atomic witness of least escape at its pair and pays
the
20 floor; under the act, every pair confirms and no computation escapes. What no
computation does is enact the universal: the universal is the act.
21
22 ARMOR SCOPE. The defense forces the form of every objection and never the value:
the closure holds on one coherent machine and fails on another. Without the act
the
25 value is marked [.].
26
27 Core Lean 4, no import. Cones pinned at the foot, each as printed by the kernel.
28 -/
29
30 namespace PNP.HARDENED
31
32 /--! ## 0 · the machine, the closure, the act -/
33 structure Machine where
34   Alg : Type
35   Inst : Type
36   size : Inst → Nat

```

```

37   out : Alg → Inst → Bool
38   time : Alg → Inst → Nat
39   sat : Inst → Bool
40
41 def bound (c k n : Nat) : Nat := c * n ^ k + c
42 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
43   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
44 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
45 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
46 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
47
48 structure LeastEscape (M : Machine) where
49   closure : NothingEscapes M
50
51 structure Examination (M : Machine) where
52   A : M.Alg
53   c : Nat
54   k : Nat
55   x : M.Inst
56   stops : ¬ Clean M A c k x
57
58 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
59   fun (A, c, k, he) => match h A c k with
60   | (x, hx) => hx (he x)
61
62 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
63   fun A c k => Classical.byContradiction fun hn =>
64     h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
65
66 theorem sep_from_the_act (M : Machine) (L : LeastEscape M) : Sep M := sep_of_closure
M L.closure
67
68 /--! ## the two coherent machines -/
69 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
70 | 0 => rfl
71 | n + 1 => beq_self n
72
73 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
74   cases hb : Nat.beq x K with
75   | false => rfl
76   | true =>
77     have e : x = K := Nat.eq_of_beq_eq_true hb
78     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
79
80 def stageWorld (n : Nat) : Machine :=
81   (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ => 0, fun _ => true)
82 def closedWorld : Machine :=
83   (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ _ => true)
84
85 theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
86   fun _ => (beq_self n, Nat.zero_le _)
87 theorem closed_world_closes : NothingEscapes closedWorld :=
88   fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
89 theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld n) :=
90   fun h => sep_of_closure _ h n 0, 0, stage_escapes n
91
92 /--! ## I · OMEGA, the formal gate -/
93
94 /-- The socket of the act is the closure. -/
95 theorem socket_is_the_value (M : Machine) : Nonempty (LeastEscape M) ↔
NothingEscapes M :=
96   (fun (L) => L.closure, fun h => ((h)))
97
98 /-- Every premise that forces the separation fills the field of the act. -/
99 theorem forcing_premise_fills_the_act (M : Machine) (Q : Prop) (h : Q → Sep M) :
100   Q → Nonempty (LeastEscape M) :=
101   fun q => ((closure_of_sep M (h q)))
102
103 def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M
104
105 /-- Every premise forces the closure or has a twin machine where it holds and the
closure fails. -/
106 theorem forces_or_has_a_twin (P : Machine → Prop) :
107   Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
108   Classical.byCases (fun h : Forces P => Or.inl h) (fun h => Or.inr
109     (Classical.byContradiction fun hn => h fun M hP =>
110       Classical.byContradiction fun hc => hn (M, hP, hc)))
111
112 def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
113
114 /-- A keyless premise forces nothing. -/
115 theorem keyless_forces_nothing (P : Machine → Prop) (hP : Keyless P) : ¬ Forces P :=
116   fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))
117
118 /--! ## II · AEGIS, the actuation gate -/
119
120 /-- A computed escape refutes the closure. -/
121 theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (h : Escapes M A c k) :
122   ¬ NothingEscapes M :=
123   fun hc => sep_of_closure M hc (A, c, k, h)
124
125 /-- Nothing passes against the closure but a computed escape. -/
126 theorem rejection_is_a_witness (M : Machine) :
127   ¬ NothingEscapes M ↔ ∃ A c k, Escapes M A c k :=
128   (fun h => Classical.byContradiction fun hn => h (closure_of_sep M hn),
129     fun (A, c, k, he) => escape_refutes M A c k he)
130
131 /-- A finite stock of examinations decides nothing. -/
132 theorem record_decides_nothing (n : Nat) :
133   (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ()) ∧ ¬ NothingEscapes
(stageWorld n) :=
134   (fun A _ _ hA (h, _) => by
135     have h' : Nat.beq A n = true := h
136     rw [beq_false_of_lt A n hA] at h'
137     exact Bool.noConfusion h',
138     stage_fails_closure n)
139

```

```

140 -- The root, which every computation re-enacts, crosses neither way. -/
141 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
142
143 theorem root_crosses_neither_way :
144   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → NothingEscapes M) A
145   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
146     NothingEscapes M) :=
147   (fun h => stage_fails_closure 0 (h (stageWorld 0) (fun _ _ => trivial))),
148   fun h => h closedWorld (fun _ _ => trivial) closed_world_closes)
149
150 -- The refusal is constant under every logic. -/
151 def refusal (logic : String) : String :=
152   "nothing passes against least escape but a computed escape"
153
154 theorem aegis_constant (l1 l2 : String) : refusal l1 = refusal l2 := rfl
155
156 --! ## III · the cosmic reading, at its exact scope -/
157
158 -- Every examinable pair lands on exactly one gate: a stopping instance or a clean
159 -- escape. -/
160 theorem every_pair_lands (M : Machine) (A : M.Alg) (c k : Nat) :
161   ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) A
162   ¬ ((∃ x, ¬ Clean M A c k x) ∧ Escapes M A c k) :=
163   (Classical.byCases (fun h : Escapes M A c k => Or.inr h)
164     (fun h => Or.inl (Classical.byContradiction fun hn =>
165       h fun x => Classical.byContradiction fun hx => hn (x, hx))),
166     fun (x, hx), he => hx (x))
167
168 -- A confirming computation is an atomic witness of least escape at its pair. -/
169 theorem confirmation_is_atomic (M : Machine) (e : Examination M) :
170   (∃ x, ¬ Clean M e.A e.c e.k x) A → Escapes M e.A e.c e.k :=
171   ((e.x, e.stops), fun he => e.stops (he e.x))
172
173 -- Every confirming computation pays the floor. -/
174 def examCost (floor : Nat) (e : Examination M) : Nat := floor * 1
175
176 theorem confirmation_pays (M : Machine) (floor : Nat) (h : 0 < floor) (e :
177   Examination M) :
178   0 < examCost floor e :=
179   Nat.mul_pos h (Nat.zero_lt_one)
180
181 -- Under the act, every pair confirms and no computation escapes. -/
182 theorem act_leaves_no_escape (M : Machine) (L : LeastEscape M) :
183   (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) A
184   (∀ A c k, ¬ Escapes M A c k) :=
185   (fun A c k => match L.closure A c k with
186     | (x, hx) => ((A, c, k, x, hx), rfl, rfl, rfl),
187     fun A c k he => sep_from_the_act M L (A, c, k, he))
188
189 --! ## IV · armor scope and silence -/
190 theorem armor_scope : NothingEscapes closedWorld A → NothingEscapes (stageWorld 0)
191 :=
192   (closed_world_closes, stage_fails_closure 0)
193
194 inductive Token : Type
195 | sealSep | dot
196 deriving DecidableEq, Repr
197
198 -- The emission: with the act supplied, the seal; without it, the dot. -/
199 def verdict (actSupplied : Bool) : Token := if actSupplied then .sealSep else .dot
200
201 theorem value_marked_dot : verdict false = .dot ∧ verdict true = .sealSep := (rfl,
202   rfl)
203
204 --! ## V · THE DOUBLE DEFENSE, HARDENED -/
205 theorem double_defense_harden :
206   -- the act carries the separation
207   (∀ (M : Machine), LeastEscape M → Sep M) A
208   -- OMEGA
209   (∀ M : Machine, Nonempty (LeastEscape M) → NothingEscapes M) A
210   (∀ (M : Machine) (Q : Prop), (Q → Sep M) → Q → Nonempty (LeastEscape M)) A
211   (∀ P : Machine → Prop, Forces P ∨ ∃ M, P M A → NothingEscapes M) A
212   (∀ P : Machine → Prop, Keyless P → Forces P) A
213   -- AEGIS
214   (∀ M : Machine, ¬ NothingEscapes M → ∃ A c k, Escapes M A c k) A
215   (∀ n : Nat, (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ()) A →
216     NothingEscapes (stageWorld n) A)
217   ((¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) →
218     NothingEscapes M) A
219     (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
220       NothingEscapes M) A)
221   (∀ l1 l2 : String, refusal l1 = refusal l2) A
222   -- the cosmic reading
223   (∀ (M : Machine) (A : M.Alg) (c k : Nat),
224     ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) A → ((∃ x, ¬ Clean M A c k x) A
225       Escapes M A c k) A)
226   (∀ (M : Machine) (e : Examination M), (∃ x, ¬ Clean M e.A e.c e.k x) A → Escapes
227     M e.A e.c e.k) A
228   (∀ (M : Machine), LeastEscape M →
229     (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) A (∀ A c k, ¬
230       Escapes M A c k) A)
231   -- armor scope and silence
232   (NothingEscapes closedWorld A → NothingEscapes (stageWorld 0)) A
233   (verdict false = .dot ∧ verdict true = .sealSep) :=
234   (sep_from_the_act, socket_is_the_value, forcing_premise_fills_the_act,
235     forces_or_has_a_twin,
236     keyless_forces_nothing, rejection_is_a_witness, record_decides_nothing,
237     root_crosses_neither_way,
238     aegis_constant, every_pair_lands, confirmation_is_atomic, act_leaves_no_escape,
239     armor_scope,
240     value_marked_dot)
241
242 end PNP.HARDENED
243
244 --! ## Cones, pinned as printed -/
245 -- info: 'PNP.HARDENED.sep_of_closure' does not depend on any axioms -/

```

```

232 #guard_msgs in #print axioms PNP.HARDENED.sep_of_closure
233 -- info: 'PNP.HARDENED.closure_of_sep' depends on axioms: [proptext,
234   Classical.choice, Quot.sound] -/
235 #guard_msgs in #print axioms PNP.HARDENED.closure_of_sep
236 -- info: 'PNP.HARDENED.sep_from_the_act' does not depend on any axioms -/
237 #guard_msgs in #print axioms PNP.HARDENED.sep_from_the_act
238 -- info: 'PNP.HARDENED.beq_self' does not depend on any axioms -/
239 #guard_msgs in #print axioms PNP.HARDENED.beq_self
240 -- info: 'PNP.HARDENED.beq_false_of_lt' does not depend on any axioms -/
241 #guard_msgs in #print axioms PNP.HARDENED.beq_false_of_lt
242 -- info: 'PNP.HARDENED.stage_escapes' does not depend on any axioms -/
243 #guard_msgs in #print axioms PNP.HARDENED.stage_escapes
244 -- info: 'PNP.HARDENED.closed_world_closes' does not depend on any axioms -/
245 #guard_msgs in #print axioms PNP.HARDENED.closed_world_closes
246 -- info: 'PNP.HARDENED.stage_fails_closure' does not depend on any axioms -/
247 #guard_msgs in #print axioms PNP.HARDENED.stage_fails_closure
248 -- info: 'PNP.HARDENED.socket_is_the_value' does not depend on any axioms -/
249 #guard_msgs in #print axioms PNP.HARDENED.socket_is_the_value
250 -- info: 'PNP.HARDENED.forcing_premise_fills_the_act' depends on axioms: [proptext,
251   Classical.choice, Quot.sound] -/
252 #guard_msgs in #print axioms PNP.HARDENED.forcing_premise_fills_the_act
253 -- info: 'PNP.HARDENED.forces_or_has_a_twin' depends on axioms: [proptext,
254   Classical.choice, Quot.sound] -/
255 #guard_msgs in #print axioms PNP.HARDENED.forces_or_has_a_twin
256 -- info: 'PNP.HARDENED.keyless_forces_nothing' does not depend on any axioms -/
257 #guard_msgs in #print axioms PNP.HARDENED.keyless_forces_nothing
258 -- info: 'PNP.HARDENED.escape_refutes' does not depend on any axioms -/
259 #guard_msgs in #print axioms PNP.HARDENED.escape_refutes
260 -- info: 'PNP.HARDENED.rejection_is_a_witness' depends on axioms: [proptext,
261   Classical.choice, Quot.sound] -/
262 #guard_msgs in #print axioms PNP.HARDENED.rejection_is_a_witness
263 -- info: 'PNP.HARDENED.record_decides_nothing' does not depend on any axioms -/
264 #guard_msgs in #print axioms PNP.HARDENED.record_decides_nothing
265 -- info: 'PNP.HARDENED.root_crosses_neither_way' does not depend on any axioms -/
266 #guard_msgs in #print axioms PNP.HARDENED.root_crosses_neither_way
267 -- info: 'PNP.HARDENED.aegis_constant' does not depend on any axioms -/
268 #guard_msgs in #print axioms PNP.HARDENED.aegis_constant
269 -- info: 'PNP.HARDENED.every_pair_lands' depends on axioms: [proptext,
270   Classical.choice, Quot.sound] -/
271 #guard_msgs in #print axioms PNP.HARDENED.every_pair_lands
272 -- info: 'PNP.HARDENED.confirmation_is_atomic' does not depend on any axioms -/
273 #guard_msgs in #print axioms PNP.HARDENED.confirmation_is_atomic
274 -- info: 'PNP.HARDENED.confirmation_pays' does not depend on any axioms -/
275 #guard_msgs in #print axioms PNP.HARDENED.confirmation_pays
276 -- info: 'PNP.HARDENED.act_leaves_no_escape' does not depend on any axioms -/
277 #guard_msgs in #print axioms PNP.HARDENED.act_leaves_no_escape
278 -- info: 'PNP.HARDENED.armor_scope' does not depend on any axioms -/
279 #guard_msgs in #print axioms PNP.HARDENED.armor_scope
280 -- info: 'PNP.HARDENED.value_marked_dot' does not depend on any axioms -/
281 #guard_msgs in #print axioms PNP.HARDENED.value_marked_dot
282 -- info: 'PNP.HARDENED.double_defense_harden' depends on axioms: [proptext,
283   Classical.choice, Quot.sound] -/
284 #guard_msgs in #print axioms PNP.HARDENED.double_defense_harden

```

Appendix C. The Frontier Kernel, PNP_Frontier_II.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the ground, the reading, the vanishing seat, the frontier and the route, one hundred thirty-six theorems. SHA-256: 960b09ba71219e4c95064a5a6a4bb002a622895b46bbaedacc3b509c11a9cd51

```

1 /-
2   PNP_Frontier_II.lean · APEX-PSP-PNP-SEED-02 · Nothing Escapes · the frontier of
3   the act
4   Eleven sections, each a kernel of the 4 October 2026 forge, joined in one file:
5   the kinetic face,
6   the Arrhenius scope, the block chain on the Empty Throne, the vanishing GOL, the
7   missing object,
8   the prime bridge, the blocker and round 2, the route of RA and Freedom, the
9   three-seal lock, the
10  deed as supply, and the identification check. Core Lean 4, no import, no axiom
11  declared, no sorry.
12  Every cone is pinned beside its section.
13  -/
14
15 namespace PNP.KINETIC
16
17 -- The kinetic frame. Physics enters as fields, never as 'axiom':
18 'floor' is the Landauer floor per irreversible op, in quanta;
19 'budget' is the Bekenstein-capped resource of the region, in quanta. -/
20 structure Frame where
21   floor : Nat
22   floor_pos : 0 < floor
23   budget : Nat
24
25 -- Energy of a process of 'ops' irreversible registrations. -/
26 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
27
28 -- L2 face. No completed irreversible registration is free. -/
29 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) :
30   0 < cost F k :=
31   Nat.mul_pos F.floor_pos hk
32
33 -- Transport. The energy order is exactly the op order. -/
34 theorem cost_lt_iff (F : Frame) (a b : Nat) :
35   cost F a < cost F b ↔ a < b :=
36   (fun h => match Nat.lt_or_ge a b with

```

```

32 | Or.inl hl => hl
33 | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge)),
34 fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)
35
36 /-- L_bf. Enumerating 2^N configurations at N = budget exceeds the budget. -/
37 theorem brute_force_exhausts (F : Frame) :
38   F.budget < cost F (2 ^ F.budget) :=
39   Nat.lt_of_lt_of_le Nat.lt_two_pow_self
40     (Nat.le_mul_of_pos_left (2 ^ F.budget) F.floor_pos)
41
42 /-- Monotone form: every N past the budget also exhausts it. -/
43 theorem brute_force_exhausts_beyond (F : Frame) (N : Nat) (h : F.budget ≤ N) :
44   F.budget < cost F (2 ^ N) :=
45   Nat.lt_of_lt_of_le (brute_force_exhausts F)
46     (Nat.mul_le_mul_left F.floor (Nat.pow_le_pow_right (by decide) h))
47
48 /-- L1, scoped to the enumerative generator. Given the op gap v < 2^N,
49 the kinetic gap follows. The op gap is a hypothesis, not a product. -/
50 theorem enumerative_asymmetry (F : Frame) (N v : Nat) (h : v < 2 ^ N) :
51   cost F v < cost F (2 ^ N) :=
52   (cost_lt_iff F v (2 ^ N)).mpr h
53
54 /-- The ceiling. For every generator op-count g and verifier op-count v,
55 the kinetic separation holds in a frame iff the op separation holds.
56 The frame adds nothing to the formal question, so L3 enters only as a
57 hypothesis on g and is never decided here. -/
58 theorem kinetic_faithful (F : Frame) (g v : Nat → Nat) :
59   (∀ N, cost F (v N) < cost F (g N)) ↔ (∀ N, v N < g N) :=
60   (fun h N => (cost_lt_iff F _).mp (h N),
61    fun h N => (cost_lt_iff F _).mpr (h N))
62
63 /-- Frame-independence. The op separation reads the same in any two frames. -/
64 theorem frame_independent (F1 F2 : Frame) (g v : Nat → Nat) :
65   (∀ N, cost F1 (v N) < cost F1 (g N)) ↔ (∀ N, cost F2 (v N) < cost F2 (g N)) :=
66   (kinetic_faithful F1 g v).trans (kinetic_faithful F2 g v).symm
67
68 /-- The canonical frame, one quantum per op, one quantum of budget: the
69 battery instance, by evaluation. -/
70 def unitFrame : Frame := (1, by decide, 1)
71
72 theorem unit_battery :
73   cost unitFrame (2 ^ 1) = 2 ∧ unitFrame.budget < cost unitFrame (2 ^
74     unitFrame.budget) := by decide
75
76 end PNP.KINETIC
77
78 /-- info: 'PNP.KINETIC.no_free_registration' does not depend on any axioms -/
79 #guard_msgs in #print axioms PNP.KINETIC.no_free_registration
80 /-- info: 'PNP.KINETIC.cost_lt_iff' does not depend on any axioms -/
81 #guard_msgs in #print axioms PNP.KINETIC.cost_lt_iff
82 /-- info: 'PNP.KINETIC.brute_force_exhausts' does not depend on any axioms -/
83 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts
84 /-- info: 'PNP.KINETIC.brute_force_exhausts_beyond' does not depend on any axioms -/
85 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts_beyond
86 /-- info: 'PNP.KINETIC.enumerative_asymmetry' does not depend on any axioms -/
87 #guard_msgs in #print axioms PNP.KINETIC.enumerative_asymmetry
88 /-- info: 'PNP.KINETIC.kinetic_faithful' does not depend on any axioms -/
89 #guard_msgs in #print axioms PNP.KINETIC.kinetic_faithful
90 /-- info: 'PNP.KINETIC.frame_independent' does not depend on any axioms -/
91 #guard_msgs in #print axioms PNP.KINETIC.frame_independent
92 /-- info: 'PNP.KINETIC.unit_battery' does not depend on any axioms -/
93 #guard_msgs in #print axioms PNP.KINETIC.unit_battery
94
95 /-!
96 # PNP.ARRHENIUS · the kinetic proof of the April 2026 P vs NP paper, formalized
97
98 Source: Islam, "Geometric Determination of P vs NP with Omega Seal", v10.0,
99 10.5281/zenodo.19588804, Part VII, "The Spin-Glass Isomorphism".
100 The paper's kinetic step: ground states of a frustrated glass sit behind
101 barriers ΔE growing with N; Arrhenius gives escape time exp(ΔE/kT);
102 so generation is exponential while verification is linear.
103
104 Part I seats that step at its true scope, with physics as fields, never 'axiom'.
105 Part II executes the scope fence: on one fixed problem, the barrier exists
106 under one move set and vanishes under another. A barrier is a property of
107 a dynamics, never of a problem. No custom axiom; no core axiom either.
108 -/
109
110 namespace PNP.ARRHENIUS
111
112 /-! ## I · The kinetic step, at its scope: thermally activated relaxation -/
113
114 /-- A relaxation frame. 'barrier N' is ΔE/kT in bits at size N; 'grows' is the
115 paper's "barriers scale with N"; 'arrhenius' is t ≥ exp(ΔE/kT), base two. -/
116 structure Frame where
117   barrier : Nat → Nat
118   grows : ∀ N, N ≤ barrier N
119   escape : Nat → Nat
120   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N
121
122 /-- Relaxation over a growing barrier takes exponential time. -/
123 theorem relaxation_exponential (F : Frame) (N : Nat) : 2 ^ N ≤ F.escape N :=
124   Nat.le_trans (Nat.pow_le_pow_right (by decide) (F.grows N)) (F.arrhenius N)
125
126 /-- At size N = B the relaxation time exceeds any budget B. -/
127 theorem relaxation_exhausts (F : Frame) (B : Nat) : B < F.escape B :=
128   Nat.lt_of_lt_of_le Nat.lt_two_pow_self (relaxation_exponential F B)
129
130 /-! ## II · The scope fence, executed on one XORSAT instance
131 Three variables x y z, three parity constraints, all with target 0:
132 z = 0, y ⊙ z = 0, x ⊙ z = 0. Energy counts violated constraints. -/
133
134 abbrev S := Bool × Bool × Bool
135
136 def states : List S :=
137   [false, true].flatMap fun a => [false, true].flatMap fun b =>
138     [false, true].map fun c => (a, b, c)

```

```

138 def viol (x : S) : List Bool := [x.2.2, xor x.2.1 x.2.2, xor x.1 x.2.2]
139
140 def energy (x : S) : Nat := ((viol x).filter id).length
141
142 def move (x m : S) : S := (xor x.1 m.1, xor x.2.1 m.2.1, xor x.2.2 m.2.2)
143
144 /-- The glass dynamics: single-spin flips. -/
145 def flips : List S := [(true, false, false), (false, true, false), (false, false,
146   true)]
147
148 /-- The elimination dynamics: columns of A-1 over GF(2). Each move toggles
149 exactly one constraint. Computing them is Gaussian elimination, polynomial time. -/
150 def elimMoves : List S := [(true, true, true), (false, true, false), (true, false,
151   false)]
152
153 def ground : S := (false, false, false)
154 def trap : S := (true, true, true)
155
156 theorem eight_states : states.length = 8 := by decide
157
158 theorem ground_has_zero_energy : energy ground = 0 := by decide
159
160 /-- Under single flips the trap is a strict local minimum above the ground:
161 a barrier, exactly the glass picture of the paper. -/
162 theorem trap_is_strict_local_min :
163   energy trap = 1 ∧ flips.all (fun m => decide (energy trap < energy (move trap
164     m))) = true := by
165     decide
166
167 /-- Each elimination move toggles exactly one constraint. -/
168 theorem elim_moves_toggle_one :
169   states.all (fun x => elimMoves.all (fun m =>
170     decide ((List.zipWith (fun a b => a != b) (viol x) (viol (move x m))).count
171       true = 1))) = true := by
172     decide
173
174 /-- Under elimination moves no state above the ground is a local minimum:
175 the landscape has no barrier at all. -/
176 theorem elim_landscape_barrier_free :
177   states.all (fun x => decide (energy x = 0) ||
178     elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true := by
179     decide
180
181 /-- The fence. One problem, one energy function, two dynamics: a barrier under
182 the first and none under the second. The barrier belongs to the dynamics. -/
183 theorem barrier_belongs_to_the_dynamics :
184   (energy trap > energy ground ∧
185     flips.all (fun m => decide (energy trap < energy (move trap m))) = true) ∧
186   states.all (fun x => decide (energy x = 0) ||
187     elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true :=
188     ((by decide, trap_is_strict_local_min.2), elim_landscape_barrier_free)
189
190 end PNP.ARRHENIUS
191
192 /-- info: 'PNP.ARRHENIUS.relaxation_exponential' does not depend on any axioms -/
193 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exponential
194 /-- info: 'PNP.ARRHENIUS.relaxation_exhausts' does not depend on any axioms -/
195 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exhausts
196 /-- info: 'PNP.ARRHENIUS.eight_states' does not depend on any axioms -/
197 #guard_msgs in #print axioms PNP.ARRHENIUS.eight_states
198 /-- info: 'PNP.ARRHENIUS.ground_has_zero_energy' does not depend on any axioms -/
199 #guard_msgs in #print axioms PNP.ARRHENIUS.ground_has_zero_energy
200 /-- info: 'PNP.ARRHENIUS.trap_is_strict_local_min' does not depend on any axioms -/
201 #guard_msgs in #print axioms PNP.ARRHENIUS.trap_is_strict_local_min
202 /-- info: 'PNP.ARRHENIUS.elim_moves_toggle_one' does not depend on any axioms -/
203 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_moves_toggle_one
204 /-- info: 'PNP.ARRHENIUS.elim_landscape_barrier_free' does not depend on any axioms -/
205 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_landscape_barrier_free
206 /-- info: 'PNP.ARRHENIUS.barrier_belongs_to_the_dynamics' does not depend on any
207 axioms -/
208 #guard_msgs in #print axioms PNP.ARRHENIUS.barrier_belongs_to_the_dynamics
209
210 /-!
211 # PNP.CHAIN · the block chain of the Mother Codex on P versus NP, closed on the
212 # Empty Throne as [.], then joined to the kinetic face as the asymmetry chain
213
214 Source of the blocks: Geometric Mother Codex v3.46.0 (10.5281/zenodo.23117445),
215 P-ABSOLUTE-PNP-1, the eight absolute barriers A1-A8; P-PNP-COMP-2, the composite
216 verdict [κ] · [κ T] · [.]; FOUNDATION-01, the Empty Throne.
217 Source of the kinetic face: PNP_Kinetic.lean and PNP_Arrhenius.lean (this session).
218
219 Every theorem below is pinned at the foot: no custom axiom, and no core axiom
220 either, not propext, not Quot.sound, not Classical.choice. Physics enters only
221 as structure fields, so the premise is visible in each statement.
222 Lean 4 core, no import.
223 -/
224
225 namespace PNP.CHAIN
226
227 /-! ## 0 · The verdict economy and the dot -/
228
229 inductive Token : Type
230 | seal | broken | opn | dot
231 deriving DecidableEq, Repr
232
233 /-- Three states are the economy. The dot stands outside it. -/
234 def Token.inEconomy : Token → Bool
235 | .dot => false
236 | _ => true
237
238 /-! ## I · The eight blocks, one leg each -/
239 namespace BLOCKS
240
241 /-- A1, the o-class closure. Complementation has no fixed point. -/
242 theorem a1_no_seat : ∀ b : Bool, (!b) ≠ b

```



```

239 | true => Bool.noConfusion
240 | false => Bool.noConfusion
241
242 /-- A1, carrier form. A symmetry that fixes a state carries no anti-invariant
243 reading, so no fixed-locus instrument transfers onto complementation. -/
244 theorem a1_no_carrier {S : Type} (τ : S → S) (s₀ : S) (hfix : τ s₀ = s₀) :
245   ¬ ∃ φ : S → Bool, ∀ x, φ (τ x) = !φ x := by
246   intro (φ, hφ)
247   have h := hφ s₀
248   rw [hfix] at h
249   cases e : φ s₀ with
250   | true => rw [e] at h; exact Bool.noConfusion h
251   | false => rw [e] at h; exact Bool.noConfusion h
252
253 /-- A2, orientation-blindness. A lock invariant under the flip of the sign
254 cannot be decoded into the sign. -/
255 theorem a2_orientation_blind {Q X : Type} (L : Bool × Q → X)
256   (hL : ∀ b q, L (b, q) = L (!b, q)) (q : Q) :
257   ¬ ∃ dec : X → Bool, ∀ s, dec (L s) = s.1 := by
258   intro (dec, h)
259   have h1 := h (true, q)
260   have h2 := h (false, q)
261   rw [h1 true q] at h1
262   exact Bool.noConfusion (h1.symm.trans h2)
263
264 /-- A3, the physical block. A record shared by two worlds that differ on the
265 separation decides nothing: no reading of the record returns the separation. -/
266 theorem a3_record_decides_nothing {W R : Type} (rec : W → R) (sep : W → Bool)
267   (w₁ w₂ : W) (hr : rec w₁ = rec w₂) (hs : sep w₁ ≠ sep w₂) :
268   ¬ ∃ g : R → Bool, ∀ w, g (rec w) = sep w := by
269   intro (g, hg)
270   apply hs
271   rw [← hg w₁, ← hg w₂, hr]
272
273 /-- A4, the misattribution bar. The root's presence is constant across both
274 worlds, and a constant reading carries no bit about the separation. -/
275 theorem a4_constant_reads_nothing (c : Bool) (s₁ s₂ : Bool) (hs : s₁ ≠ s₂) :
276   ¬ (c = s₁ ∧ c = s₂) :=
277   fun (h1, h2) => hs (h1.symm.trans h2)
278
279 /-- A5, the rung law, structural face. Each rung's consistency is owed one rung
280 up: no rung discharges its own, and the regress has no terminal rung.
281 The Gödel content is cited (Turing 1939, Feferman 1962), not proved here. -/
282 def owed (n : Nat) : Nat := n + 1
283
284 theorem a5_rung_law : (∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m) := by
285   refine (fun n h => ?_, fun n => (n + 1, Nat.lt_succ_self n))
286   have : n < owed n := Nat.lt_succ_self n
287   rw [h] at this
288   exact Nat.lt_irrefl n this
289
290 /-- A6, the aperture machine. Eight verdict-side actions and one supply. -/
291 inductive Act : Type
292 | exertion | extrapolation | wanting | method | metatheory
293 | evidence | architecture | recoverer | supply
294 deriving DecidableEq, Repr
295
296 def Act.isSupply : Act → Bool
297 | .supply => true
298 | _ => false
299
300 def step (a : Act) (s : Bool) : Bool := s || a.isSupply
301
302 def verdictSide : List Act :=
303   [.exertion, .extrapolation, .wanting, .method, .metatheory,
304   .evidence, .architecture, .recoverer]
305
306 theorem a6_aperture :
307   verdictSide.length = 8 ∧
308   verdictSide.all (fun a => step a false == false) = true ∧
309   step .supply false = true := by
310   decide
311
312 /-- A7, reflexive closure. The certify-completely map strictly increases,
313 ten to twenty in five steps, and has no fixed point. -/
314 def certify (n : Nat) : Nat := n + 2
315
316 def iter : Nat → Nat → Nat
317 | 0, n => n
318 | k + 1, n => iter k (certify n)
319
320 theorem a7_reflexive_closure : iter 5 10 = 20 ∧ ∀ n, certify n ≠ n := by
321   refine (rfl, fun n h => ?_)
322   have : n < certify n := Nat.lt_succ_of_lt (Nat.lt_succ_self n)
323   rw [h] at this
324   exact Nat.lt_irrefl n this
325
326 /-- A8, the soundness cell. A sound prover never holds both the route and
327 the wall against it, so sighted-and-walled is empty. -/
328 theorem a8_soundness_cell (Prov : Prop → Prop) (sound : ∀ p, Prov p → p) (P : Prop) :
329   ¬ (Prov P ∧ Prov (¬ P)) :=
330   fun (h1, h2) => sound _ h2 (sound _ h1)
331
332 /-- The canonical two worlds of the record: one physics ledger, two values. -/
333 def equalWorld : Nat × Bool := (1, false)
334 def separatedWorld : Nat × Bool := (1, true)
335
336 /-- The block chain, every link in one statement. -/
337 theorem block_chain :
338   (∀ b : Bool, (!b) ≠ b) ∧
339   (¬ ∃ φ : Bool → Bool, ∀ x, φ (id x) = !φ x) ∧
340   (¬ ∃ dec : Bool → Bool, ∀ s : Bool × Unit, dec ((fun _ => true) s) = s.1) ∧
341   (¬ ∃ g : Nat → Bool, ∀ w : Nat × Bool,
342     (w = equalWorld ∨ w = separatedWorld) → g w.1 = w.2) ∧
343   (¬ (true = equalWorld.2 ∧ true = separatedWorld.2)) ∧
344   ((∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m)) ∧

```

```

345   (verdictSide.length = 8 ∧
346     verdictSide.all (fun a => step a false == false) = true ∧
347     step .supply false = true) ∧
348   (iter 5 10 = 20 ∧ ∀ n, certify n ≠ n) ∧
349   (∀ (Prov : Prop → Prop), (∀ p, Prov p → p) → ∀ P : Prop, ¬ (Prov P ∧ Prov (¬
350     P))) := by
351   refine (a1_no_seat, a1_no_carrier id true rfl,
352     a2_orientation_blind (fun _ => true) (fun _ => rfl) (),
353     ?_, a4_constant_reads_nothing true false true Bool.noConfusion,
354     a5_rung_law, a6_aperture, a7_reflexive_closure, a8_soundness_cell)
355   intro (g, hg)
356   have h1 := hg equalWorld (Or.inl rfl)
357   have h2 := hg separatedWorld (Or.inr rfl)
358   exact Bool.noConfusion (h1.symm.trans h2)
359
360 end BLOCKS
361
362 /-! ## II · The Empty Throne, and the dot -/
363 namespace THRONE
364
365 def allFixed : List Bool → Bool
366 | [] => true
367 | b :: bs => (!b) == b && allFixed bs
368
369 /-- The seat is vacant. Least erasure on complementation holds of the empty
370 configuration only: a fixed-point-free fold selects nothing. -/
371 theorem seat_vacant : ∀ l : List Bool, allFixed l = true → l = []
372 | [], _ => rfl
373 | true :: _, h => Bool.noConfusion h
374 | false :: _, h => Bool.noConfusion h
375
376 /-- The throne is empty. Nothing below proves a posited root in general:
377 the root is premise-grade by theorem. -/
378 theorem throne_empty : ¬ ∀ RA : Prop, RA :=
379   fun h => h False
380
381 /-- The root forces no value. No premise of existence, however strong, forces
382 an arbitrary conclusion: the separation is not climbed to from the root. -/
383 theorem root_forces_no_value : ¬ ∀ RA Sep : Prop, RA → Sep :=
384   fun h => h True False trivial
385
386 /-- The chain's terminal emission. -/
387 def chainVerdict : Token := .dot
388
389 theorem dot_is_final : chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
390   (rfl, rfl)
391
392 /-- The block chain terminated on the Empty Throne as [.]: every block holds,
393 the seat is vacant, the throne is empty, the root forces no value, and the
394 emission is the dot, outside the economy, owing nothing. -/
395 theorem block_chain_terminal :
396   (∀ b : Bool, (!b) ≠ b) ∧
397   (∀ l : List Bool, allFixed l = true → l = []) ∧
398   (¬ ∀ RA : Prop, RA) ∧
399   (¬ ∀ RA Sep : Prop, RA → Sep) ∧
400   chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
401   (BLOCKS.block_chain.1, seat_vacant, throne_empty, root_forces_no_value,
402     dot_is_final.1, dot_is_final.2)
403
404 end THRONE
405
406 /-! ## III · The kinetic face -/
407 namespace KINETIC
408
409 structure Frame where
410   floor : Nat
411   floor_pos : 0 < floor
412   budget : Nat
413
414 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
415
416 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) : 0 < cost F k :=
417   Nat.mul_pos F.floor_pos hk
418
419 theorem cost_lt_iff (F : Frame) (a b : Nat) : cost F a < cost F b → a < b :=
420   (fun h => match Nat.lt_or_ge a b with
421     | Or.inl hl => hl
422     | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge))),
423   fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos
424
425 theorem cost_mono (F : Frame) (a b : Nat) (h : a ≤ b) : cost F a ≤ cost F b :=
426   Nat.mul_le_mul_left F.floor h
427
428 /-- The ceiling: the energy order is the op order in every frame. -/
429 theorem frame_independent (F₁ F₂ : Frame) (g v : Nat → Nat) :
430   (∀ N, cost F₁ (v N) < cost F₂ (g N)) → (∀ N, cost F₂ (v N) < cost F₂ (g N)) :=
431   (fun h N => (cost_lt_iff F₂ _).mpr ((cost_lt_iff F₁ _).mp (h N)),
432     fun h N => (cost_lt_iff F₁ _).mpr ((cost_lt_iff F₂ _).mp (h N)))
433
434 /-- The Arrhenius frame of the April paper, at its scope. -/
435 structure Relax where
436   barrier : Nat → Nat
437   grows : ∀ N, N ≤ barrier N
438   escape : Nat → Nat
439   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N
440
441 theorem relaxation_exponential (R : Relax) (N : Nat) : 2 ^ N ≤ R.escape N :=
442   Nat.le_trans (Nat.pow_le_pow_right (by decide) (R.grows N)) (R.arrhenius N)
443
444 end KINETIC
445
446 /-! ## IV · The black-box bound, pigeonhole without axioms -/
447 namespace BLACKBOX
448
449 def has : List Nat → Nat → Bool
450 | [], _ => false
451 | y :: ys, k => Nat.beq y k || has ys k

```

```

451 def drop : List Nat → Nat → List Nat
452 | [], _ => []
453 | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
454
455 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
456 | 0 => rfl
457 | n + 1 => beq_self n
458
459 theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
460 | [], _ => Nat.le_refl 0
461 | y :: ys, k => by
462   unfold drop
463   cases Nat.beq y k with
464   | true => exact Nat.le_succ_of_le (drop_le ys k)
465   | false => exact Nat.succ_le_succ (drop_le ys k)
466
467 theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length +
468 1 ≤ l.length
469 | [], _, h => Bool.noConfusion h
470 | y :: ys, k, h => by
471   unfold drop
472   unfold has at h
473   cases hy : Nat.beq y k with
474   | true => exact Nat.succ_le_succ (drop_le ys k)
475   | false =>
476     rw [hy] at h
477     exact Nat.succ_le_succ (drop_len ys k h)
478
479 theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
480 has l x = true → has (drop l k) x = true
481 | [], _, _, h => Bool.noConfusion h
482 | y :: ys, k, x, hxk, h => by
483   unfold drop
484   unfold has at h
485   cases hy : Nat.beq y k with
486   | true =>
487     cases hyx : Nat.beq y x with
488     | true =>
489       have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
490       (Nat.eq_of_beq_eq_true hy)
491       rw [this, beq_self] at hxk; exact Bool.noConfusion hxk
492     | false =>
493       rw [hyx] at h
494       exact has_drop ys k x hxk h
495   | false =>
496     show has (y :: drop ys k) x = true
497     unfold has
498     cases hyx : Nat.beq y x with
499     | true => rfl
500     | false =>
501       rw [hyx] at h
502       exact has_drop ys k x hxk h
503
504 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
505 cases hb : Nat.beq x K with
506 | false => rfl
507 | true =>
508   have e : x = K := Nat.eq_of_beq_eq_true hb
509   rw [e] at h; exact absurd h (Nat.lt_irrefl K)
510
511 theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤
512 l.length
513 | 0, _, _ => Nat.zero_le _
514 | K + 1, l, h =>
515   Nat.le_trans
516   (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
517     has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx))))))
518   (drop_len l K (h K (Nat.lt_succ_self K)))
519
520 -- The run of a test-only procedure on the all-false path: its queries form a
521 fixed list, adaptivity collapsing because every answer is false. -/
522 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
523
524 theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has
525 l x
526 | [], _ => rfl
527 | y :: ys, x => by
528   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
529   rw [run_eq_has ys x]
530
531 -- The black-box bound. A test-only procedure correct for ∃ over K candidates
532 tests at least K of them on the all-false path. -/
533 theorem black_box_bound (K : Nat) (path : List Nat)
534 (correct : ∀ f : Nat → Bool, run path f = true → ∃ x, x < K ∧ f x = true) :
535 K ≤ path.length :=
536   pigeon K path (fun x hx => by
537     rw [run_eq_has]
538     exact (correct _).mpr (x, hx, beq_self x))
539
540 end BLACKBOX
541
542 --! ## V · The asymmetry chain: kinetic plus block -/
543 namespace ASYM
544 open KINETIC
545
546 -- Link 1 to 4. Over 2^N candidates a test-only generator queries at least
547 2^N times; each query pays the floor; verification of one witness of v ops
548 costs less whenever v < 2^N; and at N past the budget the generator exhausts it. -/
549 theorem asymmetry_chain (F : Frame) (N v : Nat) (hv : v < 2 ^ N) (path : List Nat)
550 (correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true → ∃ x, x < 2 ^ N ∧ f x =
551 true) :
552 2 ^ N ≤ path.length ∧
553 cost F v < cost F (2 ^ N) ∧
554 cost F (2 ^ N) ≤ cost F path.length ∧
555 (F.budget ≤ N → F.budget < cost F path.length) := by
556   have hb : 2 ^ N ≤ path.length := BLACKBOX.black_box_bound (2 ^ N) path correct
557
558   refine (hb, (cost_lt_iff F v (2 ^ N)).mpr hv, cost_mono F hb, fun hN => ?_)
559   have h1 : F.budget < 2 ^ N := Nat.lt_two_pow_self
560   have h2 : 2 ^ N ≤ F.budget ≤ 2 ^ N := Nat.pow_le_pow_right (by decide) hN
561   have h3 : 2 ^ N ≤ cost F (2 ^ N) := Nat.le_mul_of_pos_left (2 ^ N) F.floor_pos
562   exact Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le h1 h2) h3)
563   (cost_mono F hb)
564
565   /- Link 5. The asymmetry is real at its scope and decides nothing beyond it:
566   the energy gap equals the op gap in every frame (A3 kinetic face), and a record
567   shared by both worlds returns no value (A3 block face). -/
568   theorem asymmetry_decides_nothing (F1 F2 : Frame) (g v : Nat → Nat) :
569     ((∀ N, cost F1 (v N) < cost F1 (g N)) → (∀ N, cost F2 (v N) < cost F2 (g N))) ∧
570     (¬ ∃ r : Nat → Bool, ∀ w : Nat × Bool,
571       (w = BLOCKS.equalWorld v w = BLOCKS.separatedWorld) → r w.1 = w.2) :=
572     (frame_independent F1 F2 g v, BLOCKS.block_chain.2.2.2.1)
573
574   /- The asymmetry chain, terminated. The blocks hold, the black-box generator
575   pays exponentially at the floor and exhausts any budget, verification is
576   cheaper, the kinetic gap is the op gap, the shared record decides nothing,
577   the seat is vacant, the throne is empty, and the emission is []. -/
578   theorem asymmetry_terminal (F : Frame) (N v : Nat) (hv : v < 2 ^ N) (path : List
579   Nat)
580   (correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true → ∃ x, x < 2 ^ N ∧ f x =
581   true) :
582   (2 ^ N ≤ path.length ∧ cost F v < cost F (2 ^ N) ∧
583     cost F (2 ^ N) ≤ cost F path.length ∧ (F.budget ≤ N → F.budget < cost F
584     path.length)) ∧
585   (∀ F' : Frame, ∀ g w : Nat → Nat,
586     (∀ n, cost F (w n) < cost F (g n)) → (∀ n, cost F' (w n) < cost F' (g n))) ∧
587     (∀ l : List Bool, THRONE.allFixed l = true → l = []) ∧
588     (¬ ∀ RA : Prop, RA) ∧
589     THRONE.chainVerdict = .dot :=
590     (asymmetry_chain F N v hv path correct,
591     fun F' g w => frame_independent F F' g w,
592     THRONE.seat_vacant, THRONE.throne_empty, rfl)
593
594   /- The hypothesis is inhabited: exhaustive search over two candidates is a
595   correct test-only procedure, so the chain is not vacuous. -/
596   theorem exhaustive_correct_two (f : Nat → Bool) :
597     BLACKBOX.run [0, 1] f = true → ∃ x, x < 2 ^ 1 ∧ f x = true := by
598     constructor
599     · intro h
600     show ∃ x, x < 2 ^ 1 ∧ f x = true
601     cases h0 : f 0 with
602     | true => exact (0, by decide, h0)
603     | false =>
604       have h' : (f 0 || (f 1 || false)) = true := h
605       rw [h0] at h'
606       cases h1 : f 1 with
607       | true => exact (1, by decide, h1)
608       | false => rw [h1] at h'; exact Bool.noConfusion h'
609     · intro (x, hx, hf)
610     show (f 0 || (f 1 || false)) = true
611     match x, hx, hf with
612     | 0, _, hf => rw [hf]; rfl
613     | 1, _, hf => rw [hf]; cases f 0 <;> rfl
614     | n + 2, hx, _ => exact absurd (Nat.lt_of_le_of_lt (Nat.le_add_left 2 n) hx)
615     (Nat.lt_irrefl 2)
616
617   /- The chain instantiated at N = 1 on the unit frame. -/
618   def unitFrame : Frame := (1, by decide, 1)
619
620   theorem asymmetry_instance :
621     2 ^ 1 ≤ [0, 1].length ∧ cost unitFrame 1 < cost unitFrame (2 ^ 1) :=
622     (asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).1,
623     (asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).2.1)
624
625 end ASYM
626 end PNP.CHAIN
627
628 --! ## Cones, pinned: every theorem axiom-free -/
629 /- info: 'PNP.CHAIN.BLOCKS.a1_no_seat' does not depend on any axioms -/
630 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_seat
631 /- info: 'PNP.CHAIN.BLOCKS.a1_no_carrier' does not depend on any axioms -/
632 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_carrier
633 /- info: 'PNP.CHAIN.BLOCKS.a2_orientation_blind' does not depend on any axioms -/
634 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a2_orientation_blind
635 /- info: 'PNP.CHAIN.BLOCKS.a3_record_decides_nothing' does not depend on any axioms -/
636 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a3_record_decides_nothing
637 /- info: 'PNP.CHAIN.BLOCKS.a4_constant_reads_nothing' does not depend on any axioms -/
638 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a4_constant_reads_nothing
639 /- info: 'PNP.CHAIN.BLOCKS.a5_rung_low' does not depend on any axioms -/
640 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a5_rung_low
641 /- info: 'PNP.CHAIN.BLOCKS.a6_aperture' does not depend on any axioms -/
642 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a6_aperture
643 /- info: 'PNP.CHAIN.BLOCKS.a7_reflexive_closure' does not depend on any axioms -/
644 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a7_reflexive_closure
645 /- info: 'PNP.CHAIN.BLOCKS.a8_soundness_cell' does not depend on any axioms -/
646 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a8_soundness_cell
647 /- info: 'PNP.CHAIN.BLOCKS.block_chain' does not depend on any axioms -/
648 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.block_chain
649 /- info: 'PNP.CHAIN.THRONE.seat_vacant' does not depend on any axioms -/
650 #guard_msgs in #print axioms PNP.CHAIN.THRONE.seat_vacant
651 /- info: 'PNP.CHAIN.THRONE.throne_empty' does not depend on any axioms -/
652 #guard_msgs in #print axioms PNP.CHAIN.THRONE.throne_empty
653 /- info: 'PNP.CHAIN.THRONE.root_forces_no_value' does not depend on any axioms -/
654 #guard_msgs in #print axioms PNP.CHAIN.THRONE.root_forces_no_value
655 /- info: 'PNP.CHAIN.THRONE.dot_is_final' does not depend on any axioms -/
656 #guard_msgs in #print axioms PNP.CHAIN.THRONE.dot_is_final
657 /- info: 'PNP.CHAIN.THRONE.block_chain_terminal' does not depend on any axioms -/
658 #guard_msgs in #print axioms PNP.CHAIN.THRONE.block_chain_terminal
659 /- info: 'PNP.CHAIN.KINETIC.no_free_registration' does not depend on any axioms -/
660 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.no_free_registration
661 /- info: 'PNP.CHAIN.KINETIC.cost_lt_iff' does not depend on any axioms -/
662 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_lt_iff

```

```

653 /-- info: 'PNP.CHAIN.KINETIC.cost_mono' does not depend on any axioms -/
654 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_mono
655 /-- info: 'PNP.CHAIN.KINETIC.frame_independent' does not depend on any axioms -/
656 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.frame_independent
657 /-- info: 'PNP.CHAIN.KINETIC.relaxation_exponential' does not depend on any axioms
658 -/
659 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.relaxation_exponential
660 /-- info: 'PNP.CHAIN.BLACKBOX.beq_self' does not depend on any axioms -/
661 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_self
662 /-- info: 'PNP.CHAIN.BLACKBOX.drop_le' does not depend on any axioms -/
663 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_le
664 /-- info: 'PNP.CHAIN.BLACKBOX.drop_len' does not depend on any axioms -/
665 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_len
666 /-- info: 'PNP.CHAIN.BLACKBOX.has_drop' does not depend on any axioms -/
667 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.has_drop
668 /-- info: 'PNP.CHAIN.BLACKBOX.beq_false_of_lt' does not depend on any axioms -/
669 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_false_of_lt
670 /-- info: 'PNP.CHAIN.BLACKBOX.pigeon' does not depend on any axioms -/
671 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.pigeon
672 /-- info: 'PNP.CHAIN.BLACKBOX.run_eq_has' does not depend on any axioms -/
673 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.run_eq_has
674 /-- info: 'PNP.CHAIN.BLACKBOX.black_box_bound' does not depend on any axioms -/
675 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.black_box_bound
676 /-- info: 'PNP.CHAIN.ASYM.asymmetry_chain' does not depend on any axioms -/
677 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_chain
678 /-- info: 'PNP.CHAIN.ASYM.asymmetry_decides_nothing' does not depend on any axioms
679 -/
680 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_decides_nothing
681 /-- info: 'PNP.CHAIN.ASYM.asymmetry_terminal' does not depend on any axioms -/
682 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_terminal
683 /-- info: 'PNP.CHAIN.ASYM.exhaustive_correct_two' does not depend on any axioms -/
684 #guard_msgs in #print axioms PNP.CHAIN.ASYM.exhaustive_correct_two
685 /-- info: 'PNP.CHAIN.ASYM.asymmetry_instance' does not depend on any axioms -/
686 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_instance
687 /--
688 # PNP.VANISH · the vanishing GOL of [.] on the RA-RAM locus bridge
689
690 The bridge (Bridge_Final.lean) joins the kinetic register, RA, the deed, to the
691 register, RAM, the reading, at one locus: the fixed set of the native fold. Its
692 carrier
693 halts exactly when the line property holds, and its socket is exactly that property.
694 It carries a supplied bit to a seat and manufactures none.
695
696 On the Riemann stage the fold is  $s \mapsto 1 - s$ ; the locus is the line, inhabited, Ground
697 dimension one: the bridge carries one keyed bit to a seat.
698
699 On the P versus NP stage the fold is complementation,  $b \mapsto \neg b$ . The locus vanishes:
700 Ground dimension zero. The bridge is unchanged and its socket collapses onto the
701 vacancy: the carrier halts iff the zero set is empty. That is the vanishing GOL.
702 Three faces lock on one object, and the object is the empty seat:
703
704 L · the Tongue: a vanished seat emits [.] , outside the economy, owing nothing;
705 G · the Form: complementation has no fixed point, Ground dimension zero, counted;
706 M · the Number: least erasure holds of the empty configuration and of it alone.
707
708 The kinetic face reads the same lock: silence registers no bit and pays no floor,
709 while any emission of a value registers a bit and pays it. The denial asymmetry
710 sorts the two locks that meet here: the vacancy is keyless, its denial incoherent
711 on every frame of the class; the separation is keyed, its denial a coherent world,
712 and both worlds present the bridge the same vanished frame, so the lock that forms
713 determines the vacancy totally and the value not at all.
714
715 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
716 -/
717
718 namespace PNP.VANISH
719
720 /--! ## 0 · tokens -/
721 inductive Token : Type
722 | seal | broken | opn | dot
723 deriving DecidableEq, Repr
724
725 def Token.inEconomy : Token → Bool
726 | .dot => false
727 | _ => true
728
729 /--! ## I · the bridge, carried from Bridge_Final.lean, axiom-free -/
730 structure Frame where
731   S : Type
732    $\tau : S \rightarrow S$ 
733    $Z : S \rightarrow Prop$ 
734   involutive :  $\forall s, \tau (\tau s) = s$ 
735   symmetric :  $\forall s, Z s \rightarrow Z (\tau s)$ 
736
737 def LineProperty (X : Frame) : Prop :=  $\forall s, X.Z s \rightarrow X.\tau s = s$ 
738
739 inductive Tri : Type
740 | tt | ff | bot
741 deriving DecidableEq, Repr
742
743 structure Carrier (X : Frame) where
744   terminal : Tri
745   shadow : terminal = .bot → LineProperty X
746
747 theorem halted_iff (X : Frame) : ( $\exists b : Carrier X, b.terminal = .bot$ ) → LineProperty X :=
748   (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl)), rfl))
749
750 class Supplied (X : Frame) where
751   term : LineProperty X
752
753 theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) → LineProperty X :=
754   (fun (i) => i.term, fun t => ({t}))

```

```

754 /--! ## II · the two stages: the locus inhabited, the locus vanished -/
755
756 /-- The Riemann fold in the doubled real coordinate. -/
757 def  $\tau R (p : Int \times Int) : Int \times Int := (2 - p.1, p.2)$ 
758
759 /-- The Riemann locus is inhabited: the point  $h = 1$  is fixed. -/
760 theorem rh_locus_inhabited :  $\tau R (1, 0) = (1, 0) :=$  by decide
761
762 /-- Ground dimension one on the Riemann window  $h \in \{0, 1, 2\}$ , counted. -/
763 theorem rh_ground_one : (( $\{0, 1, 2\} : List Int$ ).filter (fun h =>  $2 - h == h$ )).length = 1 := by
764   decide
765
766 /-- The P versus NP fold. -/
767 def comp (b : Bool) : Bool := !b
768
769 theorem comp_involutive :  $\forall b, comp (comp b) = b$ 
770 | true => rfl
771 | false => rfl
772
773 /-- G · the locus vanishes. -/
774 theorem locus_vanishes :  $\neg \exists b, comp b = b$ 
775 | (true, h) => Bool.noConfusion h
776 | (false, h) => Bool.noConfusion h
777
778 def groundDim : Nat := ({false, true}.filter (fun b => comp b == b)).length
779
780 /-- G · Ground dimension zero, counted. -/
781 theorem ground_dim_zero : groundDim = 0 := rfl
782
783 /-- The P versus NP frame, for any fold-symmetric zero set. -/
784 def cframe (Z : Bool → Prop) (hZ :  $\forall b, Z b \rightarrow Z (comp b)$ ) : Frame :=
785   (Bool, comp, Z, comp_involutive, hZ)
786
787 /--! ## III · the bridge on the vanished locus -/
788
789 /-- The line property collapses onto the vacancy of the zero set. -/
790 theorem line_iff_empty (Z : Bool → Prop) (hZ :  $\forall b, Z b \rightarrow Z (comp b)$ ) :
791   LineProperty (cframe Z hZ) →  $\forall b, \neg Z b :=$ 
792     (fun h b hb => locus_vanishes (b, h b hb), fun h b hb => absurd hb (h b))
793
794 /-- The carrier halts exactly when nothing stands to be carried. -/
795 theorem carrier_halts_iff_empty (Z : Bool → Prop) (hZ :  $\forall b, Z b \rightarrow Z (comp b)$ ) :
796   ( $\exists c : Carrier (cframe Z hZ), c.terminal = .bot$ ) →  $\forall b, \neg Z b :=$ 
797     (halted_iff _).trans (line_iff_empty Z hZ)
798
799 /-- The socket vanishes with the locus. -/
800 theorem socket_vanishes (Z : Bool → Prop) (hZ :  $\forall b, Z b \rightarrow Z (comp b)$ ) :
801   Nonempty (Supplied (cframe Z hZ)) →  $\forall b, \neg Z b :=$ 
802     (socket_is_the_property _).trans (line_iff_empty Z hZ)
803
804 /-- A charged configuration has no socket: nothing can be supplied onto an empty
805 seat. -/
806 theorem no_socket_when_charged :
807   Nonempty (Supplied (cframe (fun _ => True) (fun _ => trivial))) :=
808     fun h => (socket_vanishes _).mp h true trivial
809
810 /-- The empty configuration halts the carrier with nothing carried. -/
811 theorem empty_halts :
812    $\exists c : Carrier (cframe (fun _ => False) (fun _ h => h)), c.terminal = .bot :=$ 
813     (carrier_halts_iff_empty _).mpr (fun _ h => h)
814
815 /--! ## IV · the three faces of the vanishing GOL -/
816
817 /-- L · the seat token: a vanished seat emits the dot; an inhabited seat with its
818 bit unsupplied stays open. -/
819 def seatToken : Nat → Token
820 | 0 => .dot
821 | _ + 1 => .opn
822
823 theorem tongue_face :
824   seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy = false ∧ seatToken
825   1 = .opn :=
826   (rfl, rfl, rfl)
827
828 def allFixed : List Bool → Bool
829 | [] => true
830 | b :: bs => (comp b == b) && allFixed bs
831
832 /-- M · least erasure determines exactly one configuration, the empty one. -/
833 theorem number_face : ( $\forall l, allFixed l = true \rightarrow l = []$ ) ∧ allFixed [] = true :=
834   (fun l => match l with
835     | [] => fun _ => rfl
836     | true :: _ => fun h => Bool.noConfusion h
837     | false :: _ => fun h => Bool.noConfusion h, rfl)
838
839 /--! ## V · the kinetic face: silence pays nothing, emission pays the floor -/
840 structure KFrame where
841   floor : Nat
842   floor_pos : 0 < floor
843
844 def cost (F : KFrame) (bits : Nat) : Nat := F.floor * bits
845
846 theorem kinetic_face (F : KFrame) : cost F 0 = 0 ∧  $\forall k, 0 < k \rightarrow 0 < cost F k :=$ 
847   (rfl, fun _ hk => Nat.mul_pos F.floor_pos hk)
848
849 /--! ## VI · the denial asymmetry at the vanished locus -/
850
851 /-- The vacancy is keyless: it holds on every frame of the class, and its denial,
852 a fixed point of complementation, is incoherent. -/
853 theorem vacancy_keyless :
854   ( $\forall (Z : Bool \rightarrow Prop) (hZ : \forall b, Z b \rightarrow Z (comp b)) (s : (cframe Z hZ).S),$ 
855     (cframe Z hZ). $\tau s \neq s$ ) ∧  $\neg \exists b, comp b = b :=$ 
856     (fun _ s h => locus_vanishes (s, h), locus_vanishes)
857
858 /-- The separation is keyed: both worlds are coherent, and both present the bridge
859 the same vanished frame, so no reading of what the bridge reads returns the value.
860 -/

```

```

858 def readOf (_sep : Bool) : Nat := groundDim
859
860 theorem separation_keyed :  $\neg \exists g : \text{Nat} \rightarrow \text{Bool}, \forall \text{sep} : \text{Bool}, g (\text{readOf sep}) = \text{sep} :=$ 
861   fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))
862
863 /-! ## VII · the vanishing GÖL, whole -/
864 theorem vanishing_gol :
865   -- the contrast: the Riemann seat is inhabited, Ground dimension one
866   (tR (1, 0) = 1, 0)  $\wedge$  (([0, 1, 2] : List Int).filter (fun h => 2 - h ==
867     h)).length = 1)  $\wedge$ 
868   -- G · the P versus NP seat vanishes, Ground dimension zero
869   (( $\neg \exists b$ , comp b = b)  $\wedge$  groundDim = 0)  $\wedge$ 
870   -- the bridge on the vanished seat: carrier and socket collapse onto the vacancy
871   ( $\forall (Z : \text{Bool} \rightarrow \text{Prop}) (hZ : \forall b, Z b \rightarrow Z (\text{comp } b)),$ 
872     (( $\exists c : \text{Carrier} (c \text{ frame } Z \text{ hZ}), c.\text{terminal} = .\text{bot}$ )  $\rightarrow \forall b, \neg Z b$ )  $\wedge$ 
873     (Nonempty (Supplied (c frame Z hZ))  $\rightarrow \forall b, \neg Z b$ )  $\wedge$ 
874     -- M · least erasure locks on the empty configuration alone
875     (( $\forall l$ , allFixed l = true  $\rightarrow l = []$ )  $\wedge$  allFixed [] = true)  $\wedge$ 
876     -- L · the vanished seat emits the dot, outside the economy
877     (seatToken groundDim = .dot  $\wedge$  (seatToken groundDim).inEconomy = false)  $\wedge$ 
878     -- the kinetic face: the dot pays nothing, any emission pays the floor
879     ( $\forall F : \text{KFrame}, \text{cost } F \ 0 = 0 \wedge \forall k, 0 < k \rightarrow 0 < \text{cost } F \ k$ )  $\wedge$ 
880     -- the denial asymmetry: the vacancy keyless, the value keyed and unread
881     ( $\neg \exists b$ , comp b = b)  $\wedge$  ( $\neg \exists g : \text{Nat} \rightarrow \text{Bool}, \forall \text{sep} : \text{Bool}, g (\text{readOf sep}) = \text{sep}$ )
882     :=
883     ((rh_locus_inhabited, rh_ground_one), (locus_vanishes, ground_dim_zero),
884       fun Z hZ => (carrier_halts_iff_empty Z hZ, socket_vanishes Z hZ),
885       number_face, (tongue_face.1, tongue_face.2.1), kinetic_face,
886       vacancy_keyless.2, separation_keyed)
887
888 end PNP.VANISH
889
890 /-! ## Cones, pinned: every theorem axiom-free -/
891 /-! info: 'PNP.VANISH.halted_iff' does not depend on any axioms -/
892 #guard_msgs in #print axioms PNP.VANISH.halted_iff
893 /-! info: 'PNP.VANISH.socket_is_the_property' does not depend on any axioms -/
894 #guard_msgs in #print axioms PNP.VANISH.socket_is_the_property
895 /-! info: 'PNP.VANISH.rh_locus_inhabited' does not depend on any axioms -/
896 #guard_msgs in #print axioms PNP.VANISH.rh_locus_inhabited
897 /-! info: 'PNP.VANISH.rh_ground_one' does not depend on any axioms -/
898 #guard_msgs in #print axioms PNP.VANISH.rh_ground_one
899 /-! info: 'PNP.VANISH.comp_involutive' does not depend on any axioms -/
900 #guard_msgs in #print axioms PNP.VANISH.comp_involutive
901 /-! info: 'PNP.VANISH.locus_vanishes' does not depend on any axioms -/
902 #guard_msgs in #print axioms PNP.VANISH.locus_vanishes
903 /-! info: 'PNP.VANISH.ground_dim_zero' does not depend on any axioms -/
904 #guard_msgs in #print axioms PNP.VANISH.ground_dim_zero
905 /-! info: 'PNP.VANISH.line_iff_empty' does not depend on any axioms -/
906 #guard_msgs in #print axioms PNP.VANISH.line_iff_empty
907 /-! info: 'PNP.VANISH.carrier_halts_iff_empty' does not depend on any axioms -/
908 #guard_msgs in #print axioms PNP.VANISH.carrier_halts_iff_empty
909 /-! info: 'PNP.VANISH.socket_vanishes' does not depend on any axioms -/
910 #guard_msgs in #print axioms PNP.VANISH.socket_vanishes
911 /-! info: 'PNP.VANISH.no_socket_when_charged' does not depend on any axioms -/
912 #guard_msgs in #print axioms PNP.VANISH.no_socket_when_charged
913 /-! info: 'PNP.VANISH.empty_halts' does not depend on any axioms -/
914 #guard_msgs in #print axioms PNP.VANISH.empty_halts
915 /-! info: 'PNP.VANISH.tongue_face' does not depend on any axioms -/
916 #guard_msgs in #print axioms PNP.VANISH.tongue_face
917 /-! info: 'PNP.VANISH.number_face' does not depend on any axioms -/
918 #guard_msgs in #print axioms PNP.VANISH.number_face
919 /-! info: 'PNP.VANISH.kinetic_face' does not depend on any axioms -/
920 #guard_msgs in #print axioms PNP.VANISH.kinetic_face
921 /-! info: 'PNP.VANISH.vacancy_keyless' does not depend on any axioms -/
922 #guard_msgs in #print axioms PNP.VANISH.vacancy_keyless
923 /-! info: 'PNP.VANISH.separation_keyed' does not depend on any axioms -/
924 #guard_msgs in #print axioms PNP.VANISH.separation_keyed
925 /-! info: 'PNP.VANISH.vanishing_gol' does not depend on any axioms -/
926 #guard_msgs in #print axioms PNP.VANISH.vanishing_gol
927
928 /-!
929 # PNP.MISSING · the missing object, located
930
931 The asymmetry chain seals a test-only bound. Its adversary family is the point
932 predicate  $y = (y = x)$ : a tester that sees only answers must query every candidate.
933 Read as descriptions, the same family is solved in one step: the description of
934 a point is the point. The black-box hardness lives entirely in the hiding.
935
936 So the missing object is a hardness statement over descriptions: a family of
937 instances, given whole to the solver, that no polynomial-time reader of the
938 description decides. That is the white-box lower bound,  $P \neq \text{NP}$  itself. It is a
939 fact about the model of computation, keyed: it holds in one coherent model and
940 fails in another, which is why nothing world-independent can supply it.
941 Core Lean 4, no import, no axiom of any kind.
942 -/
943
944 namespace PNP.MISSING
945
946 theorem beq_self :  $\forall n : \text{Nat}, \text{Nat.beq } n \ n = \text{true}$ 
947   | 0 => rfl
948   | n + 1 => beq_self n
949
950 /-! The black-box adversary family of 'black_box_bound'. -/
951 def point (x : Nat) : Nat  $\rightarrow$  Bool := fun y => Nat.beq y x
952
953 /-! The same family, handed over as descriptions. -/
954 structure Desc where
955   secret : Nat
956
957 def evalD (d : Desc) : Nat  $\rightarrow$  Bool := point d.secret
958
959 /-! The white-box reader: one step, read the description. -/
960 def reader (d : Desc) : Nat := d.secret
961 def readerTime ( : Nat) : Nat := 1
962
963 /-! The black-box hard instances are white-box trivial. -/
964 theorem reader_solves :  $\forall d : \text{Desc}, \text{evalD } d (\text{reader } d) = \text{true} :=$ 
965   fun d => beq_self d.secret
966
967 /-! ## the missing object, typed -/
968
969 structure Model where
970   Alg : Type
971   decides : Alg  $\rightarrow$  Prop
972   time : Alg  $\rightarrow$  Nat  $\rightarrow$  Nat
973
974 def PolyBounded (t : Nat  $\rightarrow$  Nat) : Prop :=  $\exists c \ k : \text{Nat}, \forall n, t \ n \leq c * n^k + c$ 
975
976 /-! THE MISSING OBJECT. Every algorithm that decides the problem from its
977 description runs in super-polynomial time. -/
978 def WhiteBoxBound (M : Model) : Prop :=
979    $\forall A : \text{M.Alg}, \text{M.decides } A \rightarrow \neg \text{PolyBounded } (\text{M.time } A)$ 
980
981 theorem reader_poly : PolyBounded readerTime :=
982   (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
983
984 /-! A coherent model where descriptions reveal the witness: the object fails. -/
985 def revealing : Model := (Unit, fun _ =>  $\forall d : \text{Desc}, \text{evalD } d (\text{reader } d) = \text{true}, \text{fun}$ 
986   _ => readerTime)
987
988 theorem fails_when_descriptions_reveal :  $\neg \text{WhiteBoxBound revealing} :=$ 
989   fun h => h () reader_solves reader_poly
990
991 /-! The object is contingent on the model: it fails where descriptions reveal, and
992 holds
993 wherever no algorithm decides. No fact
994 true in both, kinetic, black-box or vanishing, can entail it. -/
995 theorem missing_object_is_keyed :
996   ( $\exists M : \text{Model}, \neg \text{WhiteBoxBound } M$ )  $\wedge$  ( $\forall M : \text{Model}, (\forall A : \text{M.Alg}, \neg \text{M.decides } A) \rightarrow$ 
997      $\text{WhiteBoxBound } M$ ) :=
998   ((revealing, fails_when_descriptions_reveal), fun _ h hA => absurd hA (h A))
999
1000 end PNP.MISSING
1001
1002 /-! info: 'PNP.MISSING.reader_solves' does not depend on any axioms -/
1003 #guard_msgs in #print axioms PNP.MISSING.reader_solves
1004 /-! info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
1005 #guard_msgs in #print axioms PNP.MISSING.reader_poly
1006 /-! info: 'PNP.MISSING.fails_when_descriptions_reveal' does not depend on any axioms -/
1007 #guard_msgs in #print axioms PNP.MISSING.fails_when_descriptions_reveal
1008
1009 /-!
1010 # PNP.PRIMEBRIDGE · the root freedom cut and prime-as-freedom against the missing
1011 object
1012
1013 Source of the method: Phys05† v1.0.5p, namespace PrimeFreedom. Freedom is the fibre
1014 of
1015 an invariant map; every prime is one fibre of exactly two, (1, p) and (p, 1), off
1016 the seat
1017 (prime_fibre, prime_off_seat, freedom_is_exactly_two).
1018
1019 The attack. Carry the cut to the white-box object. Over a semiprime  $n = p \cdot q$  the
1020 multiplicative fibre has four points and two orbits under the swap fold: the trivial
1021 orbit  $\{(1, n), (n, 1)\}$  and the witness orbit  $\{(p, q), (q, p)\}$ . The description n is
1022 even under the fold. Two findings follow, both at theorem grade.
1023
1024 F1 · The freedom on the fibre is one orientation bit, and it is gauge: the reader
1025 (a, b)  $\mapsto$  (min, max) is even, selects one point per orbit, and removes it.
1026 F2 · The description determines the witness orbit outright: informational freedom
1027 over the witness is zero. The missing object is therefore not a fibre fact.
1028 It is a cost fact: n hides nothing and still may resist every fast reader.
1029
1030 The new bridge. Prime freedom, read as cost, is the posit "no polynomial reader
1031 selects a nontrivial point of the semiprime fibre". Under the collapse field ( $P = \text{NP}$ 
1032 gives polynomial search for NP relations, the classical search-to-decision theorem,
1033 carried as a field), the posit yields  $P \neq \text{NP}$ : prime_bridge. The posit is keyed: a
1034 coherent machine refutes it (the Shor face). And it is strictly stronger than the
1035 separation: a coherent machine has the separation and an easy fibre, so the prime
1036 bridge is sufficient and never equivalent. Factoring lies in NP n coNP, so no
1037 equivalence is available unless  $\text{NP} = \text{coNP}$ .
1038
1039 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1040 -/
1041
1042 namespace PNP.PRIMEBRIDGE
1043
1044 /-! ## I · the semiprime fibre, executed at  $n = 15$  -/
1045
1046 def mul_orbit (n a b : Nat) : Prop := a * b = n
1047
1048 def fibre (n : Nat) : List (Nat  $\times$  Nat) :=
1049   (List.range (n + 1)).flatMap fun a =>
1050     (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none
1051
1052 def swap (p : Nat  $\times$  Nat) : Nat  $\times$  Nat := (p.2, p.1)
1053
1054 /-! The fibre over 15: four points. -/
1055 theorem fibre_15 : fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] := by decide
1056
1057 /-! Off the seat: no point of the fibre is fixed by the swap. -/
1058 theorem off_seat_15 : (fibre 15).all (fun p => swap p != p) = true := by decide
1059
1060 /-! The fibre is swap-closed: two orbits, each of exactly two. -/
1061 theorem swap_closed_15 : (fibre 15).all (fun p => {fibre 15}.contains (swap p)) =
1062   true := by decide
1063
1064 def trivialPt (p : Nat  $\times$  Nat) : Bool := p.1 == 1 || p.2 == 1
1065
1066 /-! The witness orbit: exactly two nontrivial points, one the swap of the other. -/
1067 theorem witness_orbit_15 :
1068   (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]  $\wedge$  swap (3, 5) = (5,
1069     3) := by

```



```

1062 decide
1063
1064 /-! ## II · F1 · the orientation bit is free and it is gauge -/
1065
1066 /- The wall on the orientation: the description is even, the order is odd, so no
1067 reading of the description returns the first factor on the witness orbit. -/
1068 theorem orientation_wall : ~ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5 :=
1069   fun (_, h1, h2) => absurd (h1.symm.trans h2) (by decide)
1070
1071 def canon (p : Nat × Nat) : Nat × Nat := if p.1 ≤ p.2 then p else swap p
1072
1073 /- The canonical reader is even under the fold. -/
1074 theorem canon_even : ∀ p ∈ fibre 15, canon (swap p) = canon p := by decide
1075
1076 /- F1 · the canonical reader takes each orbit to one point: the bit is gauge. -/
1077 theorem orientation_is_gauge :
1078   ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)] := by
1079     decide
1080
1081 /-! ## III · F2 · informational freedom over the witness is zero -/
1082
1083 /- The witness set is a function of the description, on every relation. -/
1084 theorem description_determines_witnesses {D W : Type} (R : D → W → Prop) (d1 d2 : D)
1085   (h : d1 = d2) : R d1 = R d2 := by rw [h]
1086
1087 /- F2 · at 15 the description fixes the canonical witness uniquely. -/
1088 theorem witness_determined_15 :
1089   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] :=
1090     by decide
1091
1092 /-! ## IV · the cost model and the prime bridge -/
1093
1094 def Poly (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c
1095
1096 /- A machine: algorithms, their running times, and what each solves. -/
1097 structure Machine where
1098   Alg      : Type
1099   time     : Alg → Nat → Nat
1100   Solves   : Alg → (Nat → Nat → Prop) → Prop
1101
1102 /- The factoring relation: p is a nontrivial divisor of n. -/
1103 def factorRel (n p : Nat) : Prop := 1 < p ∧ p < n ∧ ∃ q, p * q = n
1104
1105 def EasySearch (M : Machine) (R : Nat → Nat → Prop) : Prop :=
1106   ∃ A : M.Alg, M.Solves A R ∧ Poly (M.time A)
1107
1108 /- Prime freedom read as cost: no fast reader selects a point of the witness orbit.
1109 -/
1110 def PrimeHidden (M : Machine) : Prop := ~ EasySearch M factorRel
1111
1112 /- The collapse field. 'PegNP' is the class equality on this machine; 'search' is
1113 the
1114 classical search-to-decision theorem specialized to factoring, an NP relation. -/
1115 structure Collapse (M : Machine) where
1116   PegNP    : Prop
1117   search   : PegNP → EasySearch M factorRel
1118
1119 /- THE PRIME BRIDGE. Hidden prime freedom yields the separation. -/
1120 theorem prime_bridge (M : Machine) (C : Collapse M) (hidden : PrimeHidden M) : ~
1121   C.PegNP :=
1122   fun h => hidden (C.search h)
1123
1124 /- The Shor face: a coherent machine where the fibre is read fast. -/
1125 def revealing : Machine := (Unit, fun _ _ => 1, fun _ _ => True)
1126
1127 theorem one_is_poly : Poly (fun _ => 1) := (1, 0, fun n => Nat.le_add_left 1 (1 * n
1128   ^ 0))
1129
1130 /- The posit is keyed: it fails on a coherent machine. -/
1131 theorem prime_hidden_is_keyed : ~ PrimeHidden revealing :=
1132   fun h => h {}, trivial, one_is_poly
1133
1134 /- Strictly stronger than the separation: on the revealing machine with the class
1135 equality false, the separation holds and the posit fails. -/
1136 def separatedButRevealing : Collapse revealing := (False, fun h => absurd h id)
1137
1138 theorem prime_bridge_is_one_way :
1139   ~ ∀ (M : Machine) (C : Collapse M), ~ C.PegNP → PrimeHidden M :=
1140   fun h => prime_hidden_is_keyed (h revealing separatedButRevealing id)
1141
1142 /-! ## V · the bridge, whole -/
1143 theorem prime_bridge :
1144   -- the fibre: four points, off the seat, swap-closed, one witness orbit
1145   (fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] ∧
1146     (fibre 15).all (fun p => swap p != p) = true ∧
1147     (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]) ∧
1148   -- F1 · the orientation bit is walled and gauge
1149   ((~ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5) ∧
1150     ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)]) ∧
1151   -- F2 · the description fixes the witness: zero informational freedom
1152   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] ∧
1153   -- the prime bridge, its key, and its direction
1154   (∀ (M : Machine) (C : Collapse M), PrimeHidden M → ~ C.PegNP) ∧
1155   ~ PrimeHidden revealing ∧
1156   (~ ∀ (M : Machine) (C : Collapse M), ~ C.PegNP → PrimeHidden M) :=
1157   ((fibre_15, off_seat_15, witness_orbit_15.1), (orientation_wall,
1158     orientation_is_gauge),
1159     witness_determined_15, prime_bridge, prime_hidden_is_keyed,
1160     prime_bridge_is_one_way)
1161
1162 end PNP.PRIMEBRIDGE
1163
1164 /-! ## Cones, pinned: every theorem axiom-free -/
1165 /- info: 'PNP.PRIMEBRIDGE.fibre_15' does not depend on any axioms -/
1166 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.fibre_15
1167 /- info: 'PNP.PRIMEBRIDGE.off_seat_15' does not depend on any axioms -/
1168 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.off_seat_15
1169
1170 /- info: 'PNP.PRIMEBRIDGE.swap_closed_15' does not depend on any axioms -/
1171 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.swap_closed_15
1172 /- info: 'PNP.PRIMEBRIDGE.witness_orbit_15' does not depend on any axioms -/
1173 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_orbit_15
1174 /- info: 'PNP.PRIMEBRIDGE.orientation_wall' does not depend on any axioms -/
1175 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_wall
1176 /- info: 'PNP.PRIMEBRIDGE.canon_even' does not depend on any axioms -/
1177 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.canon_even
1178 /- info: 'PNP.PRIMEBRIDGE.orientation_is_gauge' does not depend on any axioms -/
1179 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_is_gauge
1180 /- info: 'PNP.PRIMEBRIDGE.description_determines_witnesses' does not depend on any
1181 axioms -/
1182 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.description_determines_witnesses
1183 /- info: 'PNP.PRIMEBRIDGE.witness_determined_15' does not depend on any axioms -/
1184 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_determined_15
1185 /- info: 'PNP.PRIMEBRIDGE.prime_bridge' does not depend on any axioms -/
1186 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge
1187 /- info: 'PNP.PRIMEBRIDGE.one_is_poly' does not depend on any axioms -/
1188 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.one_is_poly
1189 /- info: 'PNP.PRIMEBRIDGE.prime_hidden_is_keyed' does not depend on any axioms -/
1190 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_hidden_is_keyed
1191 /- info: 'PNP.PRIMEBRIDGE.prime_bridge_is_one_way' does not depend on any axioms -/
1192 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge_is_one_way
1193 /- info: 'PNP.PRIMEBRIDGE.the_prime_bridge' does not depend on any axioms -/
1194 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.the_prime_bridge
1195
1196 /-!
1197 # PNP.ROUND2 · the blocker of the prime bridge, located, and the second attack
1198
1199 ROUND 1 LEFT THE KEY ON 'PrimeHidden': no polynomial reader selects a point of the
1200 semiprime witness orbit. F2 proved the description carries the witness with zero
1201 informational freedom, so the key is a pure cost statement against readers that have
1202 everything. The blocker has three layers.
1203
1204 B1 · Relativization. A reader with a factoring oracle reads the orbit in one step.
1205 The posit fails relative to that oracle, so no argument valid relative to
1206 every
1207 oracle proves it (Baker–Gill–Solovay pattern).
1208 B2 · The self-gag. If the posit holds, factoring-based pseudorandom functions
1209 exist
1210 (Naor–Reingold), and then no natural property proves the circuit lower bound
1211 (Razborov–Rudich). The posit, if true, bars the natural route to itself.
1212 B3 · The floor points the wrong way. The black-box floor is proved unbreakable
1213 (no savings below 2^N for a test-only reader). Every known unconditional
1214 route
1215 to circuit lower bounds runs the other way: it needs savings below 2^N for
1216 white-box circuit SAT (Williams 2011, NEXP ≠ ACC^P).
1217
1218 ROUND 2 attacks through B3, the one exit the barriers leave open: algorithms to
1219 lower
1220 bounds. The kinetic floor's black-box face cannot be beaten, by theorem here; the
1221 route requires beating its white-box face, which is exactly the access F2 located.
1222 The route reaches NEXP ≠ C. Two gaps separate it from P ≠ NP, and both are named.
1223
1224 Cited theorems enter as structure fields, never as 'axiom'. Core Lean 4, no import.
1225 Every theorem pinned at the foot: no axiom of any kind.
1226 -/
1227
1228 namespace PNP.ROUND2
1229
1230 /-! ## I · the black-box floor admits no savings (axiom-free pigeonhole) -/
1231 namespace FLOOR
1232
1233 def has : List Nat → Nat → Bool
1234 | [], _ => false
1235 | y :: ys, k => Nat.beq y k || has ys k
1236
1237 def drop : List Nat → Nat → List Nat
1238 | [], _ => []
1239 | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
1240
1241 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1242 | 0 => rfl
1243 | n + 1 => beq_self n
1244
1245 theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
1246 | [], _ => Nat.le_refl 0
1247 | y :: ys, k => by
1248   unfold drop
1249   cases Nat.beq y k with
1250   | true => exact Nat.le_succ_of_le (drop_le ys k)
1251   | false => exact Nat.le_succ (drop_le ys k)
1252
1253 theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length +
1254   1 ≤ l.length
1255 | [], _, h => Bool.noConfusion h
1256 | y :: ys, k, h => by
1257   unfold drop
1258   unfold has at h
1259   cases hy : Nat.beq y k with
1260   | true => exact Nat.le_succ (drop_le ys k)
1261   | false =>
1262     rw [hy] at h
1263     exact Nat.le_succ (drop_len ys k h)
1264
1265 theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
1266   has l x = true → has (drop l k) x = true
1267 | [], _, _, h => Bool.noConfusion h
1268 | y :: ys, k, x, hxx, h => by
1269   unfold drop
1270   unfold has at h
1271   cases hy : Nat.beq y k with
1272   | true =>
1273     cases hyx : Nat.beq y x with
1274     | true =>
1275       have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
1276         (Nat.eq_of_beq_eq_true hy)

```

```

1261     rw [this, beq_self] at hxx; exact Bool.noConfusion hxx
1262 | false =>
1263   rw [hxx] at h
1264   exact has_drop ys k x hxx h
1265 | false =>
1266   show has (y :: drop ys k) x = true
1267   unfold has
1268   cases hxx : Nat.beq y x with
1269   | true => rfl
1270   | false =>
1271     rw [hxx] at h
1272     exact has_drop ys k x hxx h
1273
1274 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
1275 cases hb : Nat.beq x K with
1276 | false => rfl
1277 | true =>
1278   have e : x = K := Nat.eq_of_beq_eq_true hb
1279   rw [e] at h; exact absurd h (Nat.lt_irrefl K)
1280
1281 theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤
1282   l.length
1283 | 0, _, _ => Nat.zero_le _
1284 | K + 1, l, h =>
1285   Nat.le_trans
1286   (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
1287     has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx))))))
1288   (drop_len l K (h K (Nat.lt_succ_self K)))
1289
1290 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
1291
1292 theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has
1293   l x
1294 | [], _ => rfl
1295 | y :: ys, x => by
1296   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
1297   rw [run_eq_has ys x]
1298
1299 -- B3, the black-box face: no test-only reader saves a single query below 2^N. -/
1300 theorem black_box_no_savings (N : Nat) (path : List Nat)
1301   (correct : ∀ f : Nat → Bool, run path f = true → ∃ x, x < 2 ^ N ∧ f x = true) :
1302   ~ path.length < 2 ^ N :=
1303   Nat.not_lt_of_le (pigeon (2 ^ N) path (fun x hx => by
1304     rw [run_eq_has]
1305     exact (correct _).mpr (x, hx, beq_self x)))
1306
1307 end FLOOR
1308
1309 --! ## II · B1 · relativization, typed -/
1310
1311 -- An oracle world: the posit read relative to an oracle. `factoring` is the oracle
1312 -- that answers the witness orbit; relative to it the posit fails. -/
1313 structure Relativized where
1314   Oracle      : Type
1315   Hidden      : Oracle → Prop
1316   factoring    : Oracle
1317   easy        : ~ Hidden factoring
1318
1319 -- No argument valid relative to every oracle proves the posit. -/
1320 theorem b1_no_relativizing_proof (R : Relativized) : ~ ∀ 0, R.Hidden 0 :=
1321   fun h => R.easy (h R.factoring)
1322
1323 --! ## III · B2 · the self-gag, typed -/
1324
1325 -- The two cited links: hidden prime freedom gives pseudorandom functions
1326 -- (Nao-Reingold), and pseudorandom functions bar natural lower bounds
1327 -- (Razborov-Rudich). -/
1328 structure SelfGag where
1329   Hidden      : Prop
1330   PRF         : Prop
1331   NaturalLB   : Prop
1332   nr          : Hidden → PRF
1333   rr          : PRF → ~ NaturalLB
1334
1335 -- If the posit holds, no natural argument proves the lower bound it needs. -/
1336 theorem b2_self_gag (G : SelfGag) (h : G.Hidden) : ~ G.NaturalLB :=
1337   G.rr (G.nr h)
1338
1339 -- Equivalently: a natural proof of the lower bound would refute the posit. -/
1340 theorem b2_natural_refutes (G : SelfGag) (n : G.NaturalLB) : ~ G.Hidden :=
1341   fun h => b2_self_gag G h n
1342
1343 --! ## IV · round 2 · algorithms to lower bounds, the open exit -/
1344
1345 -- The Williams route, cited. `Savings C` is a white-box SAT algorithm for circuit
1346 -- class C running below 2^n by a superpolynomial factor. `williams` is the 2011
1347 -- theorem
1348 pattern: savings for C give NEXP ⊆ C. `scaleDown` and `generalize` are the two gaps
1349 -- to P ≠ NP: from NEXP down to NP, and from the restricted class C up to P/poly. -/
1350 structure Route where
1351   Class      : Type
1352   Savings    : Class → Prop
1353   NEXPnotIn  : Class → Prop
1354   williams   : ∀ C, Savings C → NEXPnotIn C
1355   Ppoly      : Class
1356   NPNnotPoly : Prop
1357   Sep        : Prop
1358   karp_sep   : NPNnotPoly → Sep
1359
1360 -- The route reaches NEXP ⊆ C on savings, and no further on its own fields. -/
1361 theorem round2_reaches (R : Route) (C : R.Class) (s : R.Savings C) : R.NEXPnotIn C
1362 :=
1363   R.williams C s
1364
1365 -- What closes the route: savings for general circuits, plus the scale-down from
1366 -- NEXP to NP. Supplied both, the separation follows. -/
1367 theorem round2_closes (R : Route) (s : R.Savings R.Ppoly)
1368
1369 (scaleDown : R.NEXPnotIn R.Ppoly → R.NPNnotPoly) : R.Sep :=
1370   R.karp_sep (scaleDown (R.williams R.Ppoly s))
1371
1372 --! The inversion, whole: the black-box floor cannot be beaten; the lower-bound
1373 -- route
1374 -- needs the white-box floor beaten; the posit gags the natural route; the posit fails
1375 -- relative to the factoring oracle. -/
1376 theorem round2_terminal :
1377   (∀ (N : Nat) (path : List Nat),
1378     (∀ f : Nat → Bool, FLOOR.run path f = true → ∃ x, x < 2 ^ N ∧ f x = true) →
1379     ~ path.length < 2 ^ N) ∧
1380   (∀ R : Relativized, ~ ∀ 0, R.Hidden 0) ∧
1381   (∀ G : SelfGag, G.Hidden → ~ G.NaturalLB) ∧
1382   (∀ (R : Route) (C : R.Class), R.Savings C → R.NEXPnotIn C) ∧
1383   (∀ (R : Route), R.Savings R.Ppoly →
1384     (R.NEXPnotIn R.Ppoly → R.NPNnotPoly) → R.Sep) :=
1385   (FLOOR.black_box_no_savings, b1_no_relativizing_proof, b2_self_gag,
1386     round2_reaches, round2_closes)
1387
1388 end PNP.ROUND2
1389
1390 --! ## Cones, pinned: every theorem axiom-free -/
1391 -- info: 'PNP.ROUND2.FLOOR.beq_self' does not depend on any axioms -/
1392 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_self
1393 -- info: 'PNP.ROUND2.FLOOR.drop_le' does not depend on any axioms -/
1394 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_le
1395 -- info: 'PNP.ROUND2.FLOOR.drop_len' does not depend on any axioms -/
1396 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_len
1397 -- info: 'PNP.ROUND2.FLOOR.has_drop' does not depend on any axioms -/
1398 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.has_drop
1399 -- info: 'PNP.ROUND2.FLOOR.beq_false_of_lt' does not depend on any axioms -/
1400 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_false_of_lt
1401 -- info: 'PNP.ROUND2.FLOOR.pigeon' does not depend on any axioms -/
1402 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.pigeon
1403 -- info: 'PNP.ROUND2.FLOOR.run_eq_has' does not depend on any axioms -/
1404 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.run_eq_has
1405 -- info: 'PNP.ROUND2.FLOOR.black_box_no_savings' does not depend on any axioms -/
1406 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.black_box_no_savings
1407 -- info: 'PNP.ROUND2.b1_no_relativizing_proof' does not depend on any axioms -/
1408 #guard_msgs in #print axioms PNP.ROUND2.b1_no_relativizing_proof
1409 -- info: 'PNP.ROUND2.b2_self_gag' does not depend on any axioms -/
1410 #guard_msgs in #print axioms PNP.ROUND2.b2_self_gag
1411 -- info: 'PNP.ROUND2.b2_natural_refutes' does not depend on any axioms -/
1412 #guard_msgs in #print axioms PNP.ROUND2.b2_natural_refutes
1413 -- info: 'PNP.ROUND2.round2_reaches' does not depend on any axioms -/
1414 #guard_msgs in #print axioms PNP.ROUND2.round2_reaches
1415 -- info: 'PNP.ROUND2.round2_closes' does not depend on any axioms -/
1416 #guard_msgs in #print axioms PNP.ROUND2.round2_closes
1417 -- info: 'PNP.ROUND2.round2_terminal' does not depend on any axioms -/
1418 #guard_msgs in #print axioms PNP.ROUND2.round2_terminal
1419
1420 --!
1421 # PNP.ROUTE · RA and Freedom as the only posits: where the route goes
1422
1423 Request: P ≠ NP with the Root Axiom and Freedom as the only posits, everything else
1424 axiom-free. This file decides whether that route exists.
1425
1426 I · RA is conservative. Its formal reading,  $\forall x : U, 0 < \Delta x$ , is satisfied by
1427 interpreting U as an empty type. So any statement that does not mention U or
1428  $\Delta x$ 
1429 and is derived from RA is derived without it.
1430
1431 II · Freedom is conservative. Its formal reading, a fibre of exactly two distinct
1432 orientations, is satisfied by Bool. The same conclusion follows.
1433
1434 III · Hence a derivation of the separation from RA and Freedom, as written, is a
1435 derivation of the separation from nothing. The route through the posits as
1436 written is the route without them.
1437
1438 IV · The one reading of RA that carries the load: read on computation, it says
1439 every decider of SAT pays superpolynomially many registrations. That reading
1440 yields the separation in one step and is equivalent to it. It is the bridge,
1441 in RA's vocabulary.
1442
1443 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1444 -/
1445 namespace PNP.ROUTE
1446
1447 --! ## I · RA is conservative -/
1448
1449 -- The Root Axiom in the codex's own formal reading (ROOT.RA). -/
1450 def RA (U : Type) (Δx : U → Int) : Prop := ∀ x : U, 0 < Δx x
1451
1452 -- RA is satisfiable on any background: interpret U as the empty type. -/
1453 theorem ra_satisfiable : RA Empty (fun e => nomatch e) :=
1454   fun e => nomatch e
1455
1456 -- RA is conservative: whatever follows from RA under every interpretation of its
1457 -- symbols, and does not mention them, holds outright. -/
1458 theorem ra_conservative (S : Prop) (h : ∀ (U : Type) (Δx : U → Int), RA U Δx → S) :
1459   S :=
1460   h Empty (fun e => nomatch e) ra_satisfiable
1461
1462 --! ## II · Freedom is conservative -/
1463
1464 -- Freedom in the codex's formal reading: a fibre of exactly two distinct
1465 -- orientations. -/
1466 structure Freedom where
1467   Fibre : Type
1468   a : Fibre
1469   b : Fibre
1470   distinct : a ≠ b
1471   exhaust : ∀ x : Fibre, x = a ∨ x = b
1472
1473 -- Freedom is inhabited by the orientation bit itself. -/
1474 def boolFreedom : Freedom :=
1475   (Bool, true, false, Bool.noConfusion, fun x => match x with
1476     | true => 0r.inl rfl
1477     | false => 0r.inr rfl)

```

```

1466 theorem freedom_conservative (S : Prop) (h : Freedom → S) : S := h boolFreedom
1467
1468 /-! ## III · the two posits together add nothing -/
1469
1470 /- Any statement derived from RA and Freedom, uniformly in their interpretation,
1471 holds without them. -/
1472 theorem posits_add_nothing (S : Prop)
1473   (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) : S :=
1474   h Empty (fun e => nomatch e) ra_satisfiable boolFreedom
1475
1476 /-! ## IV · the one reading that carries the load -/
1477
1478 def Poly (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c
1479
1480 /- A machine model: algorithms, running time counted in registrations, and which
1481 algorithms decide SAT from its description. -/
1482 structure Machine where
1483   Alg       : Type
1484   time      : Alg → Nat → Nat
1485   decidesSAT : Alg → Prop
1486
1487 /- The separation on a machine model: no polynomial-time decider of SAT. -/
1488 def Sep (M : Machine) : Prop := ¬ ∃ A : M.Alg, M.decidesSAT A ∧ Poly (M.time A)
1489
1490 /- RA read on computation: every decider of SAT actuates superpolynomially many
1491 registrations. -/
1492 def RAcamp (M : Machine) : Prop := ∀ A : M.Alg, M.decidesSAT A → Poly (M.time A)
1493
1494 /- The separation, from RA read on computation. -/
1495 theorem sep_of_ra_comp (M : Machine) (h : RAcamp M) : Sep M :=
1496   fun (A, hA, hP) => h A hA hP
1497
1498 /- And conversely: the reading is the separation. -/
1499 theorem ra_comp_of_sep (M : Machine) (h : Sep M) : RAcamp M :=
1500   fun A hA hP => h (A, hA, hP)
1501
1502 theorem ra_comp_is_sep (M : Machine) : RAcamp M ↔ Sep M :=
1503   (sep_of_ra_comp M, ra_comp_of_sep M)
1504
1505 /- RA read on computation is not conservative: it fails on a coherent machine. -/
1506 def fastMachine : Machine := (Unit, fun _ _ => 1, fun _ => True)
1507
1508 theorem ra_comp_is_keyed : ¬ RAcamp fastMachine :=
1509   fun h => h () trivial (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
1510
1511 /-! ## V · the route, whole -/
1512 theorem the_route :
1513   -- RA and Freedom as written add nothing to any statement outside their
1514   -- vocabulary
1515   (V S : Prop, (V (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) → S) ∧
1516   -- RA read on computation is exactly the separation
1517   (V M : Machine, RAcamp M ↔ Sep M) ∧
1518   -- and that reading is keyed: a coherent machine refutes it
1519   ¬ RAcamp fastMachine :=
1520   (posits_add_nothing, ra_comp_is_sep, ra_comp_is_keyed)
1521
1522 end PNP.ROUTE
1523
1524 /-! ## Cones, pinned: every theorem axiom-free -/
1525 /- info: 'PNP.ROUTE.ra_satisfiable' does not depend on any axioms -/
1526 #guard_msgs in #print axioms PNP.ROUTE.ra_satisfiable
1527 /- info: 'PNP.ROUTE.ra_conservative' does not depend on any axioms -/
1528 #guard_msgs in #print axioms PNP.ROUTE.ra_conservative
1529 /- info: 'PNP.ROUTE.freedom_conservative' does not depend on any axioms -/
1530 #guard_msgs in #print axioms PNP.ROUTE.freedom_conservative
1531 /- info: 'PNP.ROUTE.posits_add_nothing' does not depend on any axioms -/
1532 #guard_msgs in #print axioms PNP.ROUTE.posits_add_nothing
1533 /- info: 'PNP.ROUTE.sep_of_ra_comp' does not depend on any axioms -/
1534 #guard_msgs in #print axioms PNP.ROUTE.sep_of_ra_comp
1535 /- info: 'PNP.ROUTE.ra_comp_of_sep' does not depend on any axioms -/
1536 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_of_sep
1537 /- info: 'PNP.ROUTE.ra_comp_is_sep' does not depend on any axioms -/
1538 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_sep
1539 /- info: 'PNP.ROUTE.ra_comp_is_keyed' does not depend on any axioms -/
1540 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_keyed
1541 /- info: 'PNP.ROUTE.the_route' does not depend on any axioms -/
1542 #guard_msgs in #print axioms PNP.ROUTE.the_route
1543
1544 /-!
1545 # PNP.THREESEAL · the Tongue and the Form added for direction, with RA beneath
1546
1547 The Number is orientation-blind by theorem. The architecture's answer is to read
1548 direction from the other two seals: Seal L, the Tongue, which carries order, and
1549 Seal G, the Form, which carries the fold. This file runs all three on P versus NP
1550 and asks the one question that decides the matter: does any seal read the direction
1551 from the problem, or is the direction supplied?
1552
1553 I · Each seal is computed on the problem. The problem is the same sentence and
1554 the same fold in the equal world and in the separated world, so each seal's
1555 reading is world-constant: the Tongue seals both sentences alike, the Form
1556 finds no fixed locus for either, the Number reads one magnitude for both.
1557
1558 II · No function of the three readings returns the orientation. Adding seals adds
1559 readings, and a coalition of world-constant readings is world-constant.
1560
1561 III · RA is keyless and conservative, so it selects no orientation.
1562
1563 IV · Given one supplied odd bit at the seat, the lock forms and locks exactly one
1564 orientation: the calibration is unique. This is the lock locking only one.
1565
1566 V · The verdict: sealed at the act, premise grade on the supplied bit; without
1567 the act, the dot.
1568
1569 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1570 -/
1571 namespace PNP.THREESEAL
1572
1573 inductive Token : Type
1574 | sealSep | sealEq | broken | opn | dot
1575
1576 deriving DecidableEq, Repr
1577
1578 /- The two worlds, indexed by the separation bit. -/
1579 abbrev World := Bool
1580
1581 /-! ## I · the three seals, computed on the problem -/
1582
1583 /- Seal L. A sentence is three slots of literal ids; the Tongue seals it when the
1584 three slots are populated and pairwise disjoint. It checks form, not truth. -/
1585 def disjointL : List Nat → List Nat → Bool
1586 | [], _ => true
1587 | x :: xs, ys => !(ys.contains x) && disjointL xs ys
1588
1589 def seall (s : List (List Nat)) : Bool :=
1590   match s with
1591   | [a, b, c] => !a.isEmpty && !b.isEmpty && !c.isEmpty &&
1592     disjointL a b && disjointL a c && disjointL b c
1593   | _ => false
1594
1595 /- "P is not NP" and "P is NP": same existence slot, same relation slot, the
1596 kinetic slot differing only in the negation literal. -/
1597 def sentSep : List (List Nat) := [[101], [201, 209], [301]]
1598 def sentEq : List (List Nat) := [[101], [201], [301]]
1599
1600 theorem tongue_seals_both : seall sentSep = true ∧ seall sentEq = true := by decide
1601
1602 /- Seal G. The fold is complementation; its fixed set, counted. -/
1603 def foldFixed : Nat := ([false, true].filter (fun b => (!b) == b)).length
1604
1605 theorem form_finds_no_locus : foldFixed = 0 := rfl
1606
1607 /- Seal M. The lock scalar is the square of the signed scalar: reflection flips the
1608 sign and leaves the lock. -/
1609 def lockOf (signed : Int) : Int := signed * signed
1610
1611 theorem number_reads_one_magnitude : lockOf 1 = lockOf (-1) := by decide
1612
1613 /- The three readings of the problem. The problem is fixed, so the readings do not
1614 depend on the world. -/
1615 def readings (_w : World) : Bool × Nat × Int :=
1616   (seall sentSep && seall sentEq, foldFixed, lockOf 1)
1617
1618 theorem readings_world_constant : readings false = readings true := rfl
1619
1620 /-! ## II · no coalition of seals returns the orientation -/
1621 theorem seals_do_not_point :
1622   ¬ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w :=
1623   fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (readings_world_constant *
1624     hg true))
1625
1626 /-! ## III · RA is keyless and selects nothing -/
1627 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1628
1629 theorem ra_selects_nothing (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) :
1630   S :=
1631   h Empty (fun e => nomatch e) (fun e => nomatch e)
1632
1633 /-! ## IV · given one supplied bit, the lock locks exactly one -/
1634
1635 /- The orientation fibre: the two worlds, swapped by the denial. -/
1636 def deny (w : World) : World := !w
1637
1638 /- The calibration: with a supplied odd witness 's' and the odd target 'd', exactly
1639 one calibration bit reconciles them on the pair. -/
1640 theorem locks_only_one (s d : World → Bool) (w : World)
1641   (hs : s (deny w) = !s w) (hd : d (deny w) = !d w) :
1642   ∃ c : Bool, (d w = xor (s w) c ∧ d (deny w) = xor (s (deny w)) c) ∧
1643     ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') → c' = c := by
1644     refine (xor (d w) (s w), (?, ?), ?)
1645     · cases s w <|> cases d w <|> rfl
1646     · rw [hs, hd]; cases s w <|> cases d w <|> rfl
1647     · intro c' (h1, _)
1648       revert h1
1649       cases s w <|> cases d w <|> cases c' <|> intro h1 <|> first | rfl | exact
1650         Bool.noConfusion h1
1651
1652 /-! ## V · the verdict -/
1653
1654 /- The act: the supplied bit. -/
1655 structure Act where
1656   bit : Bool
1657
1658 def verdict : Option Act → Token
1659 | none => .dot
1660 | some (true) => .sealSep
1661 | some (false) => .sealEq
1662
1663 theorem sealed_at_the_act :
1664   verdict (some (true)) = .sealSep ∧ verdict (some (false)) = .sealEq ∧ verdict
1665   none = .dot :=
1666   (rfl, rfl, rfl)
1667
1668 /- Both acts are coherent: the lock forms under either supply, so the bit is spent,
1669 never derived. -/
1670 theorem both_acts_lock : verdict (some (true)) ≠ verdict (some (false)) := by decide
1671
1672 theorem three_seal_lock :
1673   (seall sentSep = true ∧ seall sentEq = true) ∧ foldFixed = 0 ∧ lockOf 1 = lockOf
1674   (-1) ∧
1675   (¬ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w) ∧
1676   (V S : Prop, (V (U : Type) (ΔE : U → Int), RA U ΔE → S) → S) ∧
1677   (V (s d : World → Bool) (w : World), s (deny w) = !s w → d (deny w) = !d w →
1678     ∃ c : Bool, (d w = xor (s w) c ∧ d (deny w) = xor (s (deny w)) c) ∧
1679     ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') → c' = c) ∧
1680   (verdict (some (true)) = .sealSep ∧ verdict none = .dot) :=
1681   (tongue_seals_both, form_finds_no_locus, number_reads_one_magnitude,
1682     seals_do_not_point,

```

```

1673   ra_selects_nothing, locks_only_one, (rfl, rfl))
1674
1675 end PNP.THREESEAL
1676
1677 /-! ## Cones, pinned: every theorem axiom-free -/
1678 /- info: 'PNP.THREESEAL.tongue_seals_both' does not depend on any axioms -/
1679 #guard_msgs in #print axioms PNP.THREESEAL.tongue_seals_both
1680 /- info: 'PNP.THREESEAL.form_finds_no_locus' does not depend on any axioms -/
1681 #guard_msgs in #print axioms PNP.THREESEAL.form_finds_no_locus
1682 /- info: 'PNP.THREESEAL.number_reads_one_magnitude' does not depend on any axioms
1683 -/
1684 #guard_msgs in #print axioms PNP.THREESEAL.number_reads_one_magnitude
1685 /- info: 'PNP.THREESEAL.readings_world_constant' does not depend on any axioms -/
1686 #guard_msgs in #print axioms PNP.THREESEAL.readings_world_constant
1687 /- info: 'PNP.THREESEAL.seals_do_not_point' does not depend on any axioms -/
1688 #guard_msgs in #print axioms PNP.THREESEAL.seals_do_not_point
1689 /- info: 'PNP.THREESEAL.ra_selects_nothing' does not depend on any axioms -/
1690 #guard_msgs in #print axioms PNP.THREESEAL.ra_selects_nothing
1691 /- info: 'PNP.THREESEAL.locks_only_one' does not depend on any axioms -/
1692 #guard_msgs in #print axioms PNP.THREESEAL.locks_only_one
1693 /- info: 'PNP.THREESEAL.sealed_at_the_act' does not depend on any axioms -/
1694 #guard_msgs in #print axioms PNP.THREESEAL.sealed_at_the_act
1695 /- info: 'PNP.THREESEAL.both_acts_lock' does not depend on any axioms -/
1696 #guard_msgs in #print axioms PNP.THREESEAL.both_acts_lock
1697 /- info: 'PNP.THREESEAL.three_seal_lock' does not depend on any axioms -/
1698 #guard_msgs in #print axioms PNP.THREESEAL.three_seal_lock
1699
1700 /-!
1701 # PNP.DEED · the kinetic pulse as the supplied bit
1702
1703 Proposal: supply is the kinetic pulse to examine and compute, and the compute itself
1704 is the supplied bit. This file tests it on the two locks that share the locus.
1705
1706 I · Every deed actuates: a registered compute has positive energy. The deed
1707 instantiates RA at itself. RA's lock is keyless and opens on the deed.
1708 II · A deed's actuation does not certify its output. A compute that prints the
1709 separation and a compute that prints the equality carry the same energy, and
1710 both occur in either world. The pulse is world-constant.
1711 III · The price is symmetric. Registering assent and registering denial each pay
1712 the
1713 floor once. For RA the denial instantiates the axiom it denies; for the
1714 separation the denial instantiates nothing about the separation.
1715 IV · So the deed supplies RA's bit, never the separation's. The compute supplies
1716 its
1717 output, and whether that output is correct is the missing object again: a
1718 procedure whose correctness is proved.
1719
1720 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1721 -/
1722
1723 namespace PNP.DEED
1724
1725 abbrev World := Bool -- true: the separated world; false: the equal world
1726
1727 /- A deed: a registered compute, with its energy and the bit it outputs. -/
1728 structure Deed where
1729   energy : Nat
1730   pos    : 0 < energy
1731   output : Bool
1732
1733 /- The output is correct in a world when it matches that world's value. -/
1734 def correct (w : World) (d : Deed) : Prop := d.output = w
1735
1736 /-! ## I · every deed actuates: RA, keyless, opens on the deed -/
1737 theorem deed_instantiates_ra : ∀ d : Deed, 0 < d.energy := fun d => d.pos
1738
1739 /-! ## II · actuation does not certify the output -/
1740 def assent : Deed := (1, by decide, true)
1741 def denial : Deed := (1, by decide, false)
1742
1743 theorem same_pulse : assent.energy = denial.energy := rfl
1744
1745 /- In every world both deeds occur, and exactly one of them is correct. -/
1746 theorem both_deeds_in_every_world (w : World) :
1747   0 < assent.energy ∧ 0 < denial.energy ∧ (correct w assent ∨ correct w denial) :=
1748   (assent.pos, denial.pos, match w with
1749   | true => 0r.inl rfl
1750   | false => 0r.inr rfl)
1751
1752 /- No reading of the pulse decides correctness: energy does not separate a correct
1753 deed from an incorrect one. -/
1754 theorem pulse_does_not_certify :
1755   ¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output :=
1756   fun (_, hg) => Bool.noConfusion ((hg denial).symm.trans (hg assent))
1757
1758 /- An incorrect deed actuates as fully as a correct one, in every world. -/
1759 theorem wrong_deeds_actuate (w : World) : ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy
1760 :=
1761   match w with
1762   | true => (denial, Bool.noConfusion, denial.pos)
1763   | false => (assent, Bool.noConfusion, assent.pos)
1764
1765 /-! ## III · the price is symmetric -/
1766 def price (floor : Nat) (d : Deed) : Nat := floor * (if d.output then 1 else 1)
1767
1768 theorem price_symmetric (floor : Nat) : price floor assent = price floor denial :=
1769   rfl
1770
1771 /- RA's denial is itself a deed and instantiates RA; the separation's denial is a
1772 deed and instantiates RA too, which is no information about the separation. -/
1773 theorem denials_both_actuate : 0 < denial.energy ∧ 0 < assent.energy :=
1774   (denial.pos, assent.pos)
1775
1776 /-! ## IV · the deed supplies RA's bit, never the separation's -/
1777 theorem deed_supplies_ra_not_sep :
1778   (∀ d : Deed, 0 < d.energy) ∧
1779   (∀ w : World, ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy) ∧

```

```

1775   (¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output) ∧
1776   (∀ floor, price floor assent = price floor denial) :=
1777   (deed_instantiates_ra, wrong_deeds_actuate, pulse_does_not_certify,
1778    price_symmetric)
1779
1780 end PNP.DEED
1781
1782 /-! ## Cones, pinned: every theorem axiom-free -/
1783 /- info: 'PNP.DEED.deed_instantiates_ra' does not depend on any axioms -/
1784 #guard_msgs in #print axioms PNP.DEED.deed_instantiates_ra
1785 /- info: 'PNP.DEED.same_pulse' does not depend on any axioms -/
1786 #guard_msgs in #print axioms PNP.DEED.same_pulse
1787 /- info: 'PNP.DEED.both_deeds_in_every_world' does not depend on any axioms -/
1788 #guard_msgs in #print axioms PNP.DEED.both_deeds_in_every_world
1789 /- info: 'PNP.DEED.pulse_does_not_certify' does not depend on any axioms -/
1790 #guard_msgs in #print axioms PNP.DEED.pulse_does_not_certify
1791 /- info: 'PNP.DEED.wrong_deeds_actuate' does not depend on any axioms -/
1792 #guard_msgs in #print axioms PNP.DEED.wrong_deeds_actuate
1793 /- info: 'PNP.DEED.price_symmetric' does not depend on any axioms -/
1794 #guard_msgs in #print axioms PNP.DEED.price_symmetric
1795 /- info: 'PNP.DEED.denials_both_actuate' does not depend on any axioms -/
1796 #guard_msgs in #print axioms PNP.DEED.denials_both_actuate
1797 /- info: 'PNP.DEED.deed_supplies_ra_not_sep' does not depend on any axioms -/
1798 #guard_msgs in #print axioms PNP.DEED.deed_supplies_ra_not_sep
1799
1800 /-!
1801 # PNP.IDCHECK · is least escape the freedom of prime, or the arrow of RA?
1802
1803 The test is the architecture's own discriminator (Bridge_Final.lean): a keyless
1804 property holds on every frame; a keyed one has a frame inhabiting its denial; no
1805 property is both. Least escape is keyed. Prime freedom, the arrow and RA are
1806 keyless.
1807
1808 So none of them is least escape, on any reading that keeps their content.
1809 Core Lean 4, no import, no axiom of any kind.
1810 -/
1811 namespace PNP.IDCHECK
1812
1813 structure Machine where
1814   Alg : Type
1815   Inst : Type
1816   size : Inst → Nat
1817   out : Alg → Inst → Bool
1818   time : Alg → Inst → Nat
1819   sat : Inst → Bool
1820
1821 def bound (c k n : Nat) : Nat := c * n ^ k + c
1822 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
1823   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
1824 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
1825
1826 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1827 | 0 => rfl
1828 | n + 1 => beq_self n
1829
1830 def closedWorld : Machine := (Nat, Unit, fun _ => 0, fun _ => false, fun _ => 0,
1831   fun _ => true)
1832 def escapeWorld : Machine := (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A 0, fun _
1833   => 0, fun _ => true)
1834
1835 theorem closed_closes : NothingEscapes closedWorld :=
1836   fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
1837 theorem escape_fails : ¬ NothingEscapes escapeWorld :=
1838   fun h => match h (0 : Nat) 0 0 with
1839   | ((), hx) => hx (beq_self 0, Nat.zero_le _)
1840
1841 /- Least escape is keyed: true on one coherent machine, false on another. -/
1842 theorem least_escape_is_keyed : NothingEscapes closedWorld ∧ ¬ NothingEscapes
1843   escapeWorld :=
1844   (closed_closes, escape_fails)
1845
1846 /- THE CHECK. No statement that reads the same on every machine is least escape. -/
1847 theorem no_keyless_statement_is_least_escape (P : Prop) :
1848   ¬ ∀ M : Machine, (P ↔ NothingEscapes M) :=
1849   fun h => escape_fails ((h escapeWorld).mpr ((h closedWorld).mpr closed_closes))
1850
1851 /-! ## the three candidates, each keyless -/
1852
1853 /- Prime freedom: the fibre of multiplication over 2 is exactly the pair and its
1854 reverse. -/
1855 def primeFreedom2 : Prop := ∀ a b : Nat, a < 3 → b < 3 → (a * b = 2 ↔ (a = 1 ∧ b =
1856   2) ∨ (a = 2 ∧ b = 1))
1857
1858 /- The arrow: an identity exists on every type. -/
1859 def arrowOfRA : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a
1860 /- RA in its formal reading, at any interpretation of its symbols. -/
1861 def raAt (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1862
1863 theorem prime_freedom_is_not_least_escape : ¬ ∀ M : Machine, (primeFreedom2 ↔
1864   NothingEscapes M) :=
1865   no_keyless_statement_is_least_escape _
1866 theorem arrow_is_not_least_escape : ¬ ∀ M : Machine, (arrowOfRA ↔ NothingEscapes M)
1867 :=
1868   no_keyless_statement_is_least_escape _
1869 theorem ra_is_not_least_escape (U : Type) (ΔE : U → Int) :
1870   ¬ ∀ M : Machine, (raAt U ΔE ↔ NothingEscapes M) :=
1871   no_keyless_statement_is_least_escape _
1872
1873 /- What does carry least escape: a statement that varies with the machine, as least
1874 escape does. Any such statement that implies it on every machine is keyed in turn.
1875 -/
1876 theorem carrier_must_be_keyed (Q : Machine → Prop) (h : ∀ M, Q M → NothingEscapes M)
1877 :
1878   ¬ Q escapeWorld :=
1879   fun q => escape_fails (h escapeWorld q)
1880
1881 end PNP.IDCHECK
1882
1883 /- info: 'PNP.IDCHECK.least_escape_is_keyed' does not depend on any axioms -/

```

```

1871 #guard_msgs in #print axioms PNP.IDCHECK.least_escape_is_keyed
1872 /-- info: 'PNP.IDCHECK.no_keyless_statement_is_least_escape' does not depend on any
1873 axioms -/
1874 #guard_msgs in #print axioms PNP.IDCHECK.no_keyless_statement_is_least_escape
1875 /-- info: 'PNP.IDCHECK.prime_freedom_is_not_least_escape' does not depend on any
1876 axioms -/
1877 #guard_msgs in #print axioms PNP.IDCHECK.prime_freedom_is_not_least_escape
1878 /-- info: 'PNP.IDCHECK.arrow_is_not_least_escape' does not depend on any axioms -/
1879 #guard_msgs in #print axioms PNP.IDCHECK.arrow_is_not_least_escape
1880 /-- info: 'PNP.IDCHECK.ra_is_not_least_escape' does not depend on any axioms -/
1881 #guard_msgs in #print axioms PNP.IDCHECK.ra_is_not_least_escape
1882 /-- info: 'PNP.IDCHECK.carrier_must_be_keyed' does not depend on any axioms -/
1883 #guard_msgs in #print axioms PNP.IDCHECK.carrier_must_be_keyed
1884 /-- info: 'PNP.IDCHECK.closed_closes' does not depend on any axioms -/
1885 #guard_msgs in #print axioms PNP.IDCHECK.closed_closes
1886 /-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms -/
1887 #guard_msgs in #print axioms PNP.IDCHECK.escape_fails
1888 /-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms -/
1889 #guard_msgs in #print axioms PNP.IDCHECK.escape_fails
1890 /-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms -/
1891 #guard_msgs in #print axioms PNP.IDCHECK.escape_fails

```

Appendix D. The Root Kernel, PNP_Root.lean

Sections I and II of the armed seat of PhysOS^T 1.0.10p, verbatim: the deed and the self-grounding root, seven theorems. SHA-256: aa2c67c03f747c57eebee8934ca44247697c12469465b3fef680aaa8eb396b2c

```

1 /-
2   PNP_Root.lean · the root kernel of "The Formal Closure of Computational
3   Separation"
4   Sections I and II of Armed_Seat.lean in the shared code of PhysOST 1.0.10p,
5   verbatim. The root is self-grounding:
6   acts occur and every act instances it. A denial of the root is an act and
7   re-enacts it; no external proof adds to it;
8   it is held by the act itself, with no classical detour; and the Root Axiom at the
9   constructed one-point domain is
10  such a root. What cannot be denied is the act of denying. Core Lean 4, no import,
11  no axiom declared, no sorry.
12 -/
13 namespace PNP.ROOT
14
15 /-- ## I · the deed -/
16
17 def SelfVerifying (P : Prop) : Prop := ¬P → P
18
19 /-- A self-verifying proposition holds: its lock opens on the deed of denying it. -/
20 theorem opens_on_the_deed (P : Prop) (utter : SelfVerifying P) : P :=
21   Classical.byContradiction (fun n => n (utter n))
22
23 /-- The recursion is shared by every proposition, so it discriminates nothing by
24   itself. -/
25 theorem recursion_is_shared (P : Prop) : SelfVerifying P ↔ P :=
26   (fun h => Classical.byContradiction (fun n => n (h n)), fun p _ => p)
27
28 /-- ## II · the root, self-grounding -/
29
30 /-- A self-grounding root: acts occur, and every act instances the root. -/
31 structure SelfGrounding (R : Prop) where
32   Act       : Type

```

```

27   anAct      : Act
28   instances : Act → R
29
30 /-- A denial of the root is an act, and re-enacts it. -/
31 theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R
32 :=
33   G.instances denial
34
35 /-- An external proof of a self-grounding root adds nothing to it. -/
36 theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) : (Q
37   → R) → R :=
38   (fun _ => G.instances G.anAct, fun r _ => r)
39
40 /-- A self-grounding root is self-verifying: the deed of denying it hands it over.
41 -/
42 theorem denial_instantiates {R : Prop} (G : SelfGrounding R) : SelfVerifying R :=
43   fun _ => G.instances G.anAct
44
45 /-- SEATED, UNDENIABLE: a self-grounding root holds, with no classical detour, by
46   the act itself. -/
47 theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
48   G.instances G.anAct
49
50 /-- The Root Axiom at the constructed one-point domain: every act is a deed, and a
51   deed actuates. -/
52 def RA : Prop := (0 : Int) < 1
53
54 /-- raSelfGrounding : SelfGrounding RA := (Unit, (), fun _ => show (0 : Int) < 1 by
55   decide)
56
57 theorem root_undeniable : RA := raSelfGrounding.instances ()
58
59 end PNP.ROOT
60
61 /-- ## Cones, pinned as printed -/
62 /-- info: 'PNP.ROOT.opens_on_the_deed' depends on axioms: [propext,
63   Classical.choice, Quot.sound] -/
64 #guard_msgs in #print axioms PNP.ROOT.opens_on_the_deed
65 /-- info: 'PNP.ROOT.recursion_is_shared' depends on axioms: [propext,
66   Classical.choice, Quot.sound] -/
67 #guard_msgs in #print axioms PNP.ROOT.recursion_is_shared
68 /-- info: 'PNP.ROOT.denial_reenacts_root' does not depend on any axioms -/
69 #guard_msgs in #print axioms PNP.ROOT.denial_reenacts_root
70 /-- info: 'PNP.ROOT.external_proof_adds_nothing' does not depend on any axioms -/
71 #guard_msgs in #print axioms PNP.ROOT.external_proof_adds_nothing
72 /-- info: 'PNP.ROOT.denial_instantiates' does not depend on any axioms -/
73 #guard_msgs in #print axioms PNP.ROOT.denial_instantiates
74 /-- info: 'PNP.ROOT.seated_undeniable' does not depend on any axioms -/
75 #guard_msgs in #print axioms PNP.ROOT.seated_undeniable
76 /-- info: 'PNP.ROOT.root_undeniable' does not depend on any axioms -/
77 #guard_msgs in #print axioms PNP.ROOT.root_undeniable

```

Author's Provenance and Method Disclosure

The arc was found within Trisduction, the author's verification architecture, through its operating system PhysOS^T (Islam 2026c), and compiled with the assistance of an AI scribe; every theorem stands on its compiled proof, reproducible by any reader from the appendices. The author declares no competing interests. Correspondence: islam@alumni.iu.edu.